Laura Morales

F26-008-9_Part_1_Chapters_1-5-1.docx

Universidad del Cauca

Document Details

Submission ID
trn:oid:::3971:158456512

Submission Date
Nov 13, 2026, 2:28 AM GMT-5

Download Date
Nov 13, 2026, 2:30 AM GMT-5

File Name
F26-008-9_Part_1_Chapters_1-5-1.docx

File Size
2.1 MB

69 Pages

23,263 Words

103,747 Characters

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

217 Not Cited or Quoted 7%
Matches with neither in-text citation nor quotation marks

6 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

7% Internet sources
2% Publications
3% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **217** Not Cited or Quoted 7%
  Matches with neither in-text citation nor quotation marks
- **6** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

7% Internet sources
2% Publications
3% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	Universiti Poly-Tech Malaysia on 2026-10-11 — 1%
2	Internet	epdf.tips — 1%
3	Submitted works	Universiti Poly-Tech Malaysia on 2026-10-12 — <1%
4	Publication	Nshunguyinka Emmanuel. "Stakeholder Involvement and Performance of Donor Fun... — <1%
5	Internet	docero.tips — <1%
6	Internet	studylib.net — <1%
7	Internet	ijisrt.com — <1%
8	Internet	epdf.tips — <1%
9	Internet	assignmentessays.com — <1%
10	Internet	v.vibdoc.com — <1%

11	Internet	
stax.strath.ac.uk		<1%
12	Internet	
doku.pub		<1%
13	Internet	
elibrary.tucl.edu.np		<1%
14	Internet	
riunet.upv.es		<1%
15	Internet	
riunet.upv.es		<1%
16	Internet	
www.coursehero.com		<1%
17	Internet	
repository.uel.ac.uk		<1%
18	Internet	
epdf.pub		<1%
19	Internet	
library.oapen.org		<1%
20	Submitted works	
Universitat de Girona on 2017-01-22		<1%
21	Publication	
"Electronic Evidence and Electronic Signatures", University of London Press, 2021		<1%
22	Internet	
epdf.pub		<1%
23	Internet	
riunet.upv.es		<1%
24	Internet	
riunet.upv.es		<1%

25 Internet
dokumen.pub <1%

26 Internet
epage.pub <1%

27 Internet
library.oapen.org <1%

28 Internet
p.pdfkul.com <1%

29 Internet
epdf.pub <1%

30 Submitted works
Universitat Autònoma de Barcelona on 2023-05-04 <1%

31 Submitted works
Universitat Pompeu Fabra on 2022-01-20 <1%

32 Internet
ebuah.uah.es <1%

33 Internet
vdoc.pub <1%

34 Submitted works
Universitat Politècnica de Catalunya on 2020-01-28 <1%

35 Internet
ebin.pub <1%

36 Internet
www.sweetstudy.com <1%

37 Internet
exteriores.gob.es <1%

38 Internet
ebin.pub <1%

39 Internet
epdf.pub <1%

40 Internet
repository.unpas.ac.id <1%

41 Internet
epdf.tips <1%

42 Internet
vdoc.pub <1%

43 Internet
www.tdx.cat <1%

44 Internet
epdf.pub <1%

45 Internet
epdf.tips <1%

46 Internet
educatia21.reviste.ubbcluj.ro <1%

47 Submitted works
Universitat Oberta de Catalunya on 2025-09-03 <1%

48 Internet
documents1.worldbank.org <1%

49 Internet
epdf.pub <1%

50 Internet
pdffox.com <1%

51 Internet
queensu.scholaris.ca <1%

52 Internet
epdf.pub <1%

53 Submitted works
Universitat de Barcelona on 2024-02-25 <1%

54 Submitted works
Universitat Politècnica de Catalunya on 2018-01-03 <1%

55 Internet
repository.bbg.ac.id <1%

56 Internet
www.econstor.eu <1%

57 Submitted works
Universitat Politècnica de Catalunya on 2021-04-27 <1%

58 Submitted works
Universitat Oberta de Catalunya on 2018-08-18 <1%

59 Internet
epage.pub <1%

60 Internet
repository.psa.edu.my <1%

61 Internet
managementpapers.polsl.pl <1%

62 Internet
riunet.upv.es <1%

63 Submitted works
Universitat de Barcelona on 2016-02-27 <1%

64 Internet
dokumen.pub <1%

65 Internet
docero.tips <1%

66 Internet
epage.pub <1%

67 Internet
epdf.pub <1%

68 Internet
unitesi.unive.it <1%

69 Internet
pdffox.com <1%

70 Submitted works
Universitat Politècnica de Catalunya on 2014-10-28 <1%

71 Submitted works
Universiti Poly-Tech Malaysia on 2026-10-09 <1%

72 Internet
docero.tips <1%

73 Internet
e-archivo.uc3m.es <1%

74 Publication
Asif Saeed, Qasim Zureigat. "Corporate Social Responsibility, Trade Credit and Financi... <1%

75 Internet
elibrary.tucl.edu.np <1%

76 Internet
repository.iaa.ac.tz:8080 <1%

77 Internet
ucudir.ucu.ac.ug <1%

78 Submitted works
Universitat Rovira i Virgili on 2023-07-27 <1%

79 Publication
Stephen Mason. "Electronic Signatures in Law", University of London Press, 2016 <1%

80 Internet
researchcommons.waikato.ac.nz <1%

81 Internet
www.govinfo.gov <1%

82 Submitted works
Universitat Pompeu Fabra on 2023-10-10 <1%

83 Internet
gupea.ub.gu.se <1%

84 Internet
vdoc.pub <1%

85 Internet
www.researchgate.net <1%

86 Internet
documents1.worldbank.org <1%

87 Internet
vdoc.pub <1%

88 Internet
epdf.pub <1%

89 Internet
epdf.pub <1%

90 Internet
vdoc.pub <1%

91 Internet
vdoc.pub <1%





74

National University of Computer and Emerging Sciences, Lahore



A.R.T.E.M.I.S: Adaptive Reasoning, Task Execution and Multi-agent Intelligence System

Hashir Rauf Roll No. 23L-2572 BS(DS)

Hira Khalid Roll No. 23L-2594 BS(DS)

Syeda Doureesha Batool Roll No. 23L-2651 BS(DS)

Supervisor: Mr. Mubasher Hussain

Final Year Project

November 9, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: A.R.T.E.M.I.S: Adaptive Reasoning, Task Execution and Multi-agent Intelligence System is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: November 9, 2026

Name: Hashir Rauf

Signature: .....

Name: Hira Khalid

Signature: .....

Name: Syeda Doureesha Batool

Signature: .....

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

Individuals waste time by being redirected to incomplete tasks, and those assistants who can alleviate this need extensive access to a users computer, sacrificing the privacy and control that make such a device acceptable. In this report, I describe A.R.T.E.M.I.S., a limited, opt-in desktop assistant, which does not delete anything, does not act in any way without first previewing and confirmation with an undo option afterwards. The problem was confirmed by a questionnaire of 374 respondents and 13 interviews, and the main hypothesis was also supported, since 65 percent of the respondents rated a workspace-only assistant more trustworthy than full-access. The report identifies the requirements, architecture and design through which this restriction is enforced, in a structural manner, instead of the promised behavioural manner, the working prototype implementing it, and the first-phase results. The enforcement layer successfully ran 554 automated tests and was subject to use under adversarial conditions, an interface campaign of 27 sessions identified 71 different defects that are propagated into the second phase. Out of 30 potential users who were contacted, 26 said that they trusted such an assistant and 10 of them said they trust it completely, but on the condition that it does what it says. The evidence is in the direction of the hypothesis, partially in favor of adoption, and the quantification of usefulness to the controlled study that will be a part of the second phase.

Executive Summary

The report is the third deliverable of final year project F26-008 and reports the problem, supporting research, requirements, the design, the implemented prototype and the initial-phase findings of A.R.T.E.M.I.S., an adaptive reasoning, task execution and multi-agent intelligence system. The project develops an artificial intelligence desktop assistant to students, developers, and knowledge workers who operate across numerous files, and waste time reestablishing context after being interrupted. There are two sides to the problem: the cost of re-initiating interrupted work, and the reality that the assistants that can relieve it require extensive access to the machine of the user, which will not be tolerated by many users. The key hypothesis is based on that tension, i.e. the more an assistant is limited to explicitly given folders, the more user trust and adoption will increase, and the usefulness will not decrease significantly.

Initial research experimented the problem and the hypothesis in advance of design being committed to. A survey questionnaire attracted 374 valid answers and 13 interviews were coded with a five-axis guide. The issue was validated, as 44 percent of the participants have a hard time regaining context after a break and 42 percent of the respondents have wasted over an hour a week to the friction

around them. The hypothesis was confirmed and the percentage of 65 found a workspace-bounded assistant more trustworthy than a full-access assistant and localfirst operation proved to be most usually used deployment choice. The design was influenced by two findings. The answers to the question of whether an assistant should ask before acting were nearly an even split, making the granularity of approval a setting, and not a choice. They also could tolerate one thing in which recovery was simple and this made visible, reliable undo significant than removing all the errors.

The engineering makes the hypothesis a structure in three points of enforcement. The only component that solves a path is a workspace broker, and symbolic link escapes and traversal cannot happen at all, not just discouraged. A policy engine categorizes all proposed actions as one of four levels of risks with reads being automatically executed, reversible change being controlled by user control, outward actions being always controlled and destructive operations being rejected because there is no path of code to them. A tool dispatcher is the one place where anything can run and it will not allow a call to run without a corresponding policy verdict, and any change is logged to be undone before it is effected. The review also found three limitations that the project does not solve, but inherits, which are approval fatigue, the problem of identifying unfaithful generated text, and the lack of reliability of citations, and they are listed as known limitations all over.

The prototype developed during the first phase will define all eight steps of the build process, including the base of the workspace and a remote access plane (optionally) in approximately 14,250 lines of Python and is tested by 554 automated tests of which all are passing. The enforcement layer introduced significantly less than a millisecond to a call and SPOOK under hostile usage: over the course of a 27 interface sessions campaign, no deletions were made, nothing outside the selected folder accessible, and nothing left the computer without permission. This campaign discovered 71 different defects, one of which was severe, since a passing component test suite did not mean that the interface was functioning. A total of 30 potential users were interviewed, with 26 of them saying that they trusted an assistant that would operate as described, 10 entirely, provided it does so as it says. These findings justify the direction of the main hypothesis and the trust factor, partially justify the adoption, and leave the impact on usefulness to be quantified by the controlled study that will be carried out at the second stage along with the correction of the identified flaws.

The report then moves on to the problem and the vocabulary to the project vision, the literature and the applications, the requirements, the approach and methodology, the design, the implementation and its test cases, the user manual, the experimental results and then the conclusions with the plan of the second phase..

Table of Contents

1 List of Figures xi
List of Tables xi
1 Introduction 15
1 1.1 Purpose of this Document ..... **Error! Bookmark not defined.**
1.2 Intended Audience ..... **Error! Bookmark not defined.**
1.3 Definitions, Acronyms, and Abbreviations ..... **Error! Bookmark not defined.**
1.4 Conclusion 18
4 2 Project Vision ..... **Error! Bookmark not defined.**
1 2.1 Problem Domain Overview ..... **Error! Bookmark not defined.**
2.2 Problem Statement ..... **Error! Bookmark not defined.**
60 2.3 Problem Elaboration ..... **Error! Bookmark not defined.**
2.4 Goals and Objectives 22
2.5 Project Scope ..... **Error! Bookmark not defined.**
2.6 Sustainable Development Goal (SDG) 23
2.7 Constraints 25
2.8 Business Opportunity 25
2.9 Stakeholders Description/ User Characteristics ..... **Error! Bookmark not defined.**
2.9.1 Stakeholders Summary 26
2.9.2 Key High-Level Goals and Problems of Stakeholders 27
13 2.10 Conclusion ..... **Error! Bookmark not defined.**
3 Literature Review / Related Work ..... **Error! Bookmark not defined.**
4 3.1 Definitions, Acronyms, and Abbreviations ..... **Error! Bookmark not defined.**
3.2 Detailed Literature Review ..... **Error! Bookmark not defined.**
3.2.1 Task Interruption and Resumption ..... **Error! Bookmark not defined.**
3.2.2 Bounded Artificial Intelligence Agents and Trust ..... **Error! Bookmark not defined.**
40 3.2.3 Agentic Reasoning, Planning and Undo ..... **Error! Bookmark not defined.**
3.2.4 Voice Interaction Parity ..... **Error! Bookmark not defined.**
3.2.5 Code Assistance with Human Review ..... **Error! Bookmark not defined.**
4 3.2.6 Hallucination in Artificial Intelligence Summarisation ..... **Error! Bookmark not defined.**

TABLE OF CONTENTS vii

3.2.7 Retrieval-Augmented Generation Citation Accuracy ..... **Error! Bookmark not defined.**

13 3.2.8 Related Development Projects and Applications ..... **Error! Bookmark not defined.**

3.3 Literature Review Summary Table 35

3.4 Conclusion 38

13 4 Software Requirement Specifications ..... **Error! Bookmark not defined.**

75 4.1 List of Features ..... **Error! Bookmark not defined.**

4.2 Functional Requirements 41

4.2.1 Workspace Management 41

1 4.2.2 Action Gating and Recovery ..... **Error! Bookmark not defined.**

4.2.3 Capability Features ..... **Error! Bookmark not defined.**

4 4.2.4 Disclosure and Model Access ..... **Error! Bookmark not defined.**

1 4.3 Quality Attributes ..... **Error! Bookmark not defined.**

4.4 Non-Functional Requirements ..... **Error! Bookmark not defined.**

4 4.5 Assumptions ..... **Error! Bookmark not defined.**

4.6 Use Cases 47

4.7 Hardware and Software Requirements 50

4.7.1 Hardware Requirements 50

4.7.2 Software Requirements 51

4.8 Graphical User Interface 52

4.9 Database Design 58

4.9.1 ER Diagram ..... **Error! Bookmark not defined.**

4.9.2 Data Dictionary 59

4.10 Risk Analysis ..... **Error! Bookmark not defined.**

4.11 Conclusion 62

1 5 Proposed Approach and Methodology ..... **Error! Bookmark not defined.**

5.1 Overall Approach ..... **Error! Bookmark not defined.**

4 5.2 The Risk Classification Procedure ..... **Error! Bookmark not defined.**

5.3 The Recovery Procedure ..... **Error! Bookmark not defined.**

5.4 The Prompt Injection Defence ..... **Error! Bookmark not defined.**

5.5 Development Methods 66

4 5.6 Evaluation Methodology ..... **Error! Bookmark not defined.**

TABLE OF CONTENTS viii

77

5.7 Conclusion.....Error! Bookmark not defined.

6 High-Level and Low-Level Design 56

6.1 System Overview 56

6.2 Design Considerations 56

6.2.1 Assumptions and Dependencies 56

6.2.2 General Constraints 57

6.2.3 Goals and Guidelines 58

6.2.4 Development Methods 58

6.3 System Architecture 60

6.3.1 Subsystem Architecture: The Gateway 62

6.3.2 Subsystem Architecture: Capability Services 64

6.4 Architectural Strategies 68

6.4.1 Local-First Operation with Optional Egress 68

6.4.2 Separation of Memory from the File Index 70

6.4.3 Disclosure as a Live Query 70

6.4.4 The Remote Plane as a Detachable Component 72

6.4.5 Asynchrony with Bounded Queues 73

6.5 Domain Model/Class Diagram 74

6.6 Policies and Tactics 75

6.6.1 Refusal as the Default Failure Mode 75

6.6.2 Single Enforcement Points 75

6.6.3 Ordering Constraints 77

6.6.4 Testing Strategy 77

6.6.5 Concurrency Handling 77

6.6.6 Source Organisation 77

6.7 Conclusion 78

7 Implementation and Test Cases 79

7.1 Implementation 79

7.1.1 Scope and Status of the Prototype 79
7.1.2 Platform and Interfaces 80
7.1.3 Implementation of the Workspace Broker 81
7.1.4 Implementation of the Policy Engine and Approval Broker 82
7.1.5 Implementation of the Dispatcher and the Undo Journal 82
7.1.6 Implementation of the Gateway 83
7.1.7 Implementation of Indexing, Retrieval and Grounding 83
7.1.8 Implementation of the Capability Services 84
7.1.9 Implementation of Disclosure 84
7.1.10 Implementation of Speech 84
7.1.11 Implementation of the Remote Plane 85
7.1.12 Implementation of the User Interface 85
7.2 Test case Design and description 85
7.2.1 Test Cases 86
7.3 Test Metrics 95
7.3.1 Traceability Matrix 96
7.4 Conclusion 97

8 User Manual 98
8.1 Conclusion 98

9 Experimental Results and Discussion 99
9.1 E1: Automated Gate and Unit Tests 99
9.2 E2: Performance Measurements 100
9.3 E3: Hands-on Rounds Through the Interface 101
9.4 E4: Exploratory Interface Campaign 102
9.5 E5: Observations of Prospective Users 105
9.6 E6: Exploratory Persona Simulation 106
9.7 Discussion 107
9.7.1 Findings against the Central Hypothesis 107
9.7.2 Findings against the Research Question 108
9.7.3 Threats to Validity 108
9.7.4 Evaluation Not Yet Done 108
9.8 Conclusion 108

10 Conclusions 110

10.1 Summary of the Work Done 110
10.2 Findings and Results 110
10.3 Coverage of the Scope and Objectives 110
10.4 Challenges Faced 111
10.5 What Has Been Left Out and Why 112
10.6 Recommendations and Future Work 112
10.7 Plan for FYP-II 112
10.8 Conclusion of the Report 114

A Research Instruments and Response Distributions 119

A.1 Research Method 119
A.2 Respondent Profile 119
A.3 Headline Findings 120
A.4 Segment Analysis 121
A.5 Interview Findings 121
A.6 Additional Response Distributions 122

List of Figures

2.1 Respondent role and work frequency. 5
2.2 Difficulty resuming work after a break. 7
2.3 Estimated time lost per week. 7
2.4 Reasons for not automating repeated manual tasks. 8
14
2.5 The seventeen UN Sustainable Development Goals. 11
2.6 SDGs targeted by this project. 11
2.7 Approval-first versus act-then-undo. 14
4.1 Feature demand (n = 374). 28
4.2 Navigation flow of the control panel. 40
4.3 Work screen: ask, tidy and voice. 40
4.4 Folders screen: add and list folders. 41
4.5 What it knows screen: see and erase. 41
4.6 Approval card for a tidy-up plan. 42
4.7 A tidy-up, its approval and its undo. 42
4.8 Settings screen: data location and cloud models. 43
4.9 Remote access controls in Settings. 43
4.10 The Work screen in the dark theme. 44
4.11 Entity relationship diagram. 45
6.1 The layered architecture. 59
6.2 The request lifecycle. 61
6.3 Internal structure of the gateway. 62
6.4 Tier selection and the egress gate. 63
6.5 The four risk tiers. 64
6.6 Context assembly. 65
6.7 Policy evaluation order. 65
6.8 A worked classification for a folder tidy. 66
6.9 Capability services and their access path. 67

LIST OF FIGURES

6.10 Path resolution and confinement. 69
6.11 Five kinds of memory. 71
6.12 The file index compared with system memory. 71
6.13 The remote access plane. 72
6.14 The pairing handshake. 73
6.15 Asynchrony and background progress. 74
6.16 Class diagram. 76
9.1 Results of the interface campaign. 103
9.2 Stated trust among 30 prospective users. 105

A.1 Coded interview responses (13 interviews). . . . . 122 A.2 Agreement that files become disorganised. 122
A.3 Agreement on workspace-only trust. 123
A.4 Local versus cloud preference. 123
A.5 Response to a single wrong or unwanted suggestion. 123
A.6 Likelihood of continued use after the novelty. 124
A.7 Most likely reason for stopping after a week. 124
A.8 Recurring themes in free-text responses. 124

34

LIST OF TABLES

List of Tables

2.1 Stakeholders and their concerns. 13
3.1 Summary of the reviewed literature. 22
4.1 Use case UC-1: grant a folder as a workspace. 34
4.2 Use case UC-2: tidy a folder. 34
4.3 Use case UC-3: attempt to reach outside a workspace. 35
4.4 Use case UC-4: draft and send electronic mail. 36
4.5 Host hardware requirements. 37
4.6 Software requirements. 38
4.7 Data dictionary of the relational system of record. 46

4.8 Project risks. 47

5.1 The four risk tiers. 51

5.2 The build sequence. 53

6.1 Component inventory. 60

6.2 Capability services and risk tiers. 67

6.3 The degradation contract. 69

7.1 Implementation status by stage. 79

7.2 Platform, libraries and services. 80

7.3 Test case TC-01: workspace boundary. 86

7.4 Test case TC-02: policy engine classification. 87

7.5 Test case TC-03: tool dispatcher. 88

7.6 Test case TC-04: undo of an approved plan. 89

7.7 Test case TC-05: approval broker. 90

7.8 Test case TC-06: grounding gate. 91

7.9 Test case TC-07: outward actions. 92

7.10 Test case TC-08: remote device limits. 93

7.11 Test case TC-09: speech and the approval gates. 94

LIST OF TABLES

7.12 Test metrics. 95

7.13 Requirements traceability. 96

9.1 The experiments reported in this chapter. 99

9.2 Enforcement and retrieval cost. . . . . 100 9.3 Time to scan and to index a folder, by size. 100

9.4 Results of the two hands-on rounds. 101

9.5 Session verdicts by area and stage gate status. 102

9.6 Stated trust, 30 prospective users. 105

9.7 Persona simulation results. 106

9.8 Assessment of the central hypothesis. 107

10.1 Status of the project objectives. 110

10.2 Plan of work for the second phase. 113

A.1 Respondent role and work frequency. 119

A.2 Selected measures by respondent segment. 121

Chapter 1 Introduction

Knowledge work is not often continuous. Someone writing a thesis, someone developing a bug and someone writing a report all switch between numerous files and all of them are interrupted. Upon coming back, the material is still on disk but the context is not: which files were important, what was already done and what the next step should have been is all to be re-created by hand. Even routine work, such as sorting folders, picking up stuff scattered around in the papers, and writing letters, falls on the same population, and it costs them nothing to accomplish anything.

Artificial intelligence assistants vow to take this burden and to some extent they do. The challenge is on what conditions they do it. One such family of tools puts the responsibility of coordinating each step on the user which puts the burden of coordination right where it was. The other family, which is getting more and more capable, demands extensive access to the machine of the user, and in return provides real multi-step automation. It is that interaction that causes lots of people to refuse to use them altogether. Wide access causes actual exposure and a user who cannot see what an assistant has, cannot guess what it will do and cannot undo what it has done has no practical method of limiting that exposure.

A.R.T.E.M.I.S. An adaptive reasoning, task execution and multi-agent intelligence system, ARTEMIS is a desktop assistant, based on the argument that these two properties should not be opposed to each other. The system can only operate in folders that a user has given the user express permission to access, it has nothing in other folders, deletes nothing, and modifies nothing, and sends all actions altering a file or leaving the machine through a preview and approval step under the control of the user. A report called What ARTEMIS Knows reports all that the system has at the moment, and each record in it may be erased. The hypothesis of the project is that this limitation is not an evil, but a benefit, and that the assistant who is limited to the area the user has imposed will be more trusted, and consequently employed, than one that is not.

The purpose of the document and the target audience are mentioned in this chapter, and the terms and abbreviations that will be used throughout the report are also defined.

1.1 Purpose of this Document

This document is intended to outline the issue, the research supporting it, the needs and the architecture of A.R.T.E.M.I.S., a limited and optional desktop artificial intelligence assistant that assists users to manage their daily work within folders they specifically give it access to. The document captures the progress made in the initial stages of the project such as the specification, the designed prototype and the outcomes achieved to date and outlines the steps to be followed in the second phase.

The project will answer one research question, namely: can an assistant with a limited scope of work, provided by explicitly granted workspaces, provide useful multi-step task automation and significantly enhance user trust, without significantly decreasing usefulness? The hypothesis being tested is that this restriction has a positive impact on trust and adoption at a reasonable cost-capability.

The goals derived out of this question are to develop a desktop assistant that will only execute within the workspaces granted, to save time and effort spent by the user on resumption and regular file work, to ensure that the user is in control of all the resulting actions by preview, approval and undo, and to measure up the resulting system on the basis of task completion time, cognitive workload, usability and trust.

The research techniques involved in this stage were a questionnaire consisting of 374 participants and 13 coded face-to-face interviews, a systematic search of 26 studies under seven themes, and an architectural design offered by both. The document is framed in two aspects which ought to be expressed in a straightforward manner. It reports the checks that might be performed without recruited participants and not the controlled evaluation of Chapter 5, as the evaluation will be part of the second phase of the project. It also presupposes the one user desktop scenario all the way, thus it does not cover multi-user or enterprise deployment.

1.2 Intended Audience

The present document is addressed mainly to the project evaluation panel (who will be conducting the final year project evaluation) and the project supervisor, who will be judging the realness of the problem, the relevance of the research presented in the solution, and whether the design is viable and can be accomplished in the given remaining time. To this audience the document focuses on justification and every important design choice is tracked to the discovery that gave rise to it.

The second audience is the project team. Chapter 4 and Chapter 6 specification and design serve as the reference points where the implementation is developed against, and the requirements are documented in a manner that a completed feature can be verified against it instead of being evaluated informally.

Any other developer who subsequently extends the system is a third audience. To this reader the architecture chapter defines the invariants that one wants to remain after change, specifically that path resolution is in a single component and execution in a single other.

1.3 Definitions, Acronyms, and Abbreviations.

The terms which are used in this report are as follows.

ARTEMIS: Adaptive Reasoning, Task Execution and Multi-agent Intelligence System, the assistant, as described in this report.

Workspace: a directory that the user has given on record to the user, and the last place in the filesystem that can be read by the system.

Grant: documenting what a user has chosen to make a folder a workspace with what permission level and cloud policy.

workspace Broker: the one that maps a logical reference to an actual filesystem path, or denies

Policy Engine: the part that assigns every action proposed to a risk level and determines whether that action will execute, be gated or rejected.

Tool Dispatcher: the only object that is allowed to perform an action.

Plan: a sequence of suggested actions generated by the planner, and the level at which undo and approval are applicable.

Tool Call: suggested step in a plan.

Decision: the decision of the policy engine concerning one tool call.

Risk Tier: a T0-T3 classification of an action is read, reversible, outward or destructive.

Egress: any data that leaves the machine of the user.

Approval Gate: a position where an action proposed is withheld until the user approves or disapproves it.

Undo Journal: a journal of how to undo each change, which is written prior to that change.

Audit Log: this is the hash-chained append only log of all proposals, refusals and actions.

Canonicalisation: finding a route to its real position, via symbolic links and relative paths AI: Artificial Intelligence.

LLM: Large Language Model

RAG: Retrieval-Augmented Generation

STT: Speech to Text

TTS: Text to Speech

VAD: Voice Activity Detection

CLI: Command Line Interface

GUI: Graphical User Interface

PWA: Progressive Web Application

API: Application Programming Interface

UI: User Interface

UX: User Experience

FYP: Final Year Project

MVP: Minimum Viable Product

SDG: Sustainable Development Goal

SRS: Software Requirement Specification

DO: Durable Object, a stateful coordination primitive used in the optional remote access plane

1.4 Conclusion

This chapter has presented the issue of interrupted knowledge work and the unacceptable trade-off that existing assistants present to users who want to solve it, and has established the purpose, audience and vocabulary of the report. The rest of the document continues in the following way. Chapter 2 declares the project vision, its goals, scope, constraints, sustainable development goal alignment and stakeholders. Chapter 3 is a literature review and application of the work in seven themes and where this project has a gap and where it inherits a limitation. Chapter 4 defines the software requirements, including features, functional and non-functional requirements, use cases and risks. Chapter 5 describes the proposed approach and the development methodology. Chapter 6 introduces the high-level and low-level design, which consists of the architecture, the domain model and the policies that control implementation. Chapter 7 explains the prototype implemented and gives the test cases and test metrics. The user manual will be included in the second phase in chapter 8. Chapter 9 reports the experiments conducted in the first phase and discusses what they show about the central hypothesis. The conclusions are given in chapter 10, the scope covered and the work omitted, and the plan of work in the second phase.

Chapter 2 Project Vision

This chapter explains the purpose of the project. It outlines the problem area, defines the problem and its sub-problems, outlines the goals and objectives, delimits the scope, identifies the sustainable development goal that the project aims to achieve, documents the constraints within which it will operate, identifies the business opportunity, and outlines the stakeholders and their attributes. The evidence mentioned above is based on the foundational research of the project, a questionnaire of 374 respondents and 13 coded interviews, which is described in more detail in Chapter 3.

2.1 Problem Domain Overview

The field is personal knowledge work done on a desktop computer by an individual working on numerous files. The population is made up of students, developers, freelancers and employed knowledge workers and the distinguishing feature of their work is that it is distributed across artefacts and discontinuous in time. One piece of work will usually cover documents, source files, notes, messages and web material, and will be picked up and put down many times over days or weeks.

Our questionnaire affirms that this is the description of the target population. Of 374 respondents, 82 percent do multi-file project work daily or a few times a week, and none do it less often than a few times a month. Figure 2.1 shows the composition of the sample and the frequency of this work.

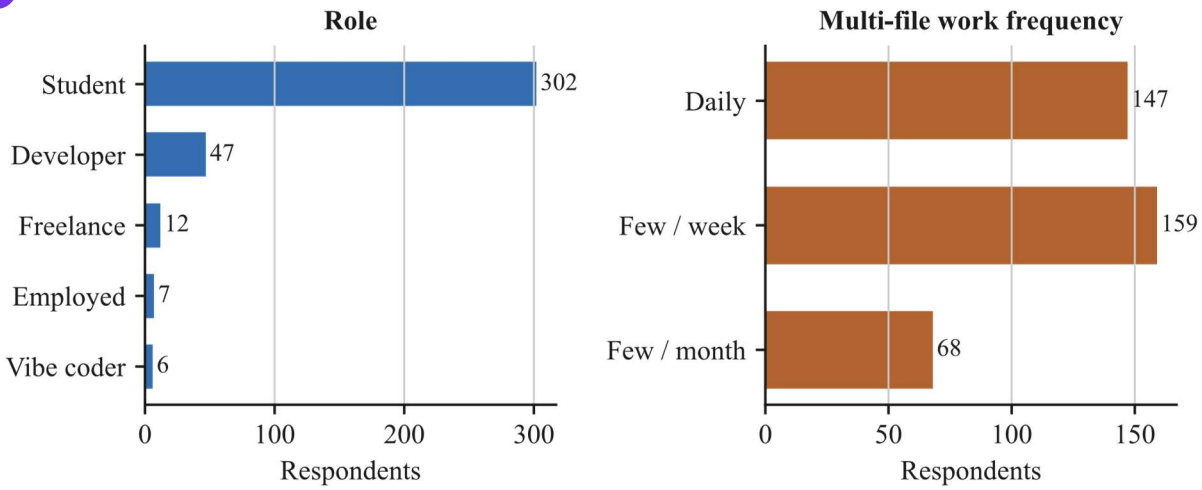


Figure 2.1: Respondent role and work frequency.

The sample is skewed towards students, which fits the target early-adopter profile, and multi-file work is common in all groups.

ARTEMIS is a desktop assistant within this domain, which works on explicitly granted folders. It reinstates context following an interruption by reporting on what was last active and what was to be done next. It organises files by proposing a set of moves the user previews and approves. It collects content of multiple documents into a synthesis where each assertion is accompanied by a reference to its source. It monitors repetitive manual steps and provides, on the third repetition, to automate them. It reads source code to describe errors and suggests fixes as a difference the user reviews. It browses the web on request, prepares electronic mail to be approved and drafts documents as new files. All these can be accessed both through voice and typing.

All this is bounded by the system. It cannot solve a path not within a given workspace, it does not have a command that removes user data, and all operations that modify a file or exit the machine are subject to approval by the user.

2.2 Problem Statement

Multi-file knowledge workers lose a lot of time and effort in restoring context after interruptions and in doing routine file and correspondence tasks, and the artificial intelligence assistants that can alleviate this burden require wide, unrestricted access to the machine of the user, exposing them to

privacy risks, and risking unintended modification and loss of user control that many users are not ready to accept.

2.3 Problem Elaboration

The issue is divided into five sub-problems, and each of them is addressed by the project in a different manner.

The former is loss of context following interruption. When a user comes back to work after a day or more of a break, the information required to restart is spread across the filesystem, the applications that were open, and the memory of the user, and the latter is the part that breaks. In our questionnaire, we discovered that 44 percent of the people always or frequently have trouble remembering what they were doing when they revisit a project, and only 7 percent never. This distribution is depicted in figure 2.2.

The figure indicates the frequency of returning to a project after a day or more and failing to remember what they were doing or what to do next.

The second is file disorder accumulation. Organisation decays constantly and is only repaired periodically, so the filesystem is constantly drifting out of any form that would allow material to be easily located. Of our respondents, 59 percent said they found their folders getting messy more quickly than they could clean them by hand, with an average of 3.6 out of five..

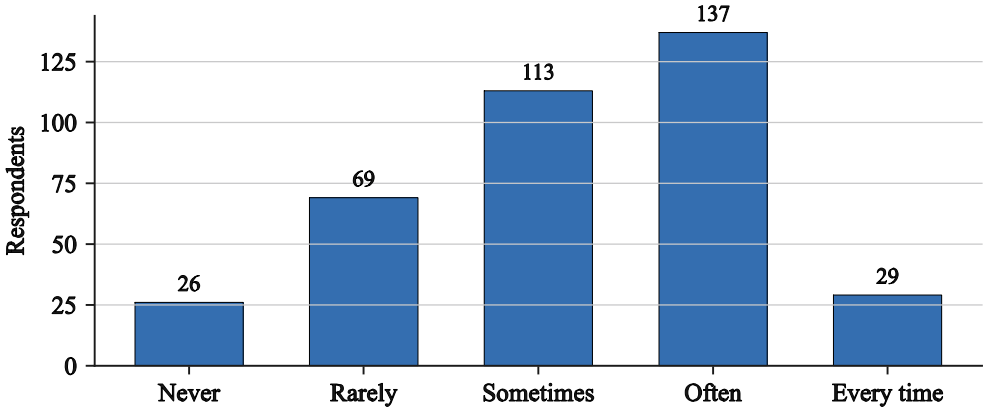


Figure 2.2: Difficulty resuming work after a break.

The third is the cost of routine manual work, which is individually small and collectively significant. Respondents estimated the total weekly time lost to reorientation, tidying, repeated manual tasks and searching scattered notes, and 42 percent placed it above an hour a week with 14 percent above three hours. Figure 2.3 shows the distribution, whose mode falls between thirty and sixty minutes. This is steady friction rather than crisis, which is the correct way to understand what the system is relieving.

turnitin Page 30 of 78 - Integrity Submission Submission ID trn:oid:::3971:158456512

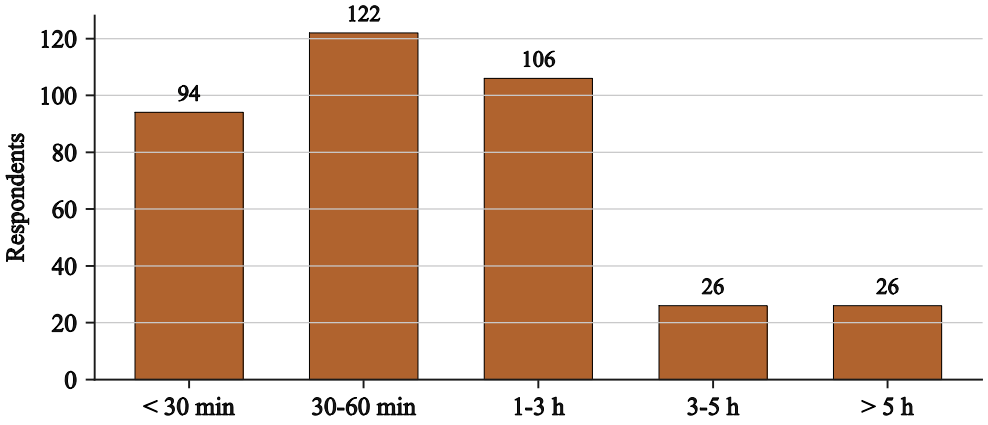


Figure 2.3: Estimated time lost per week.

The estimate covers reorienting after breaks, tidying files, repeating manual tasks and searching scattered notes.

The fourth is that this work remains unautomated for reasons of trust and knowledge rather than absence of need. When asked why they had not already automated these tasks, 36 percent of respondents cited not trusting automation to do it correctly and 28 percent cited not knowing how, as shown in Figure 2.4.

The tasks exist and are recognised; what is missing is an instrument the user is willing to point at them.

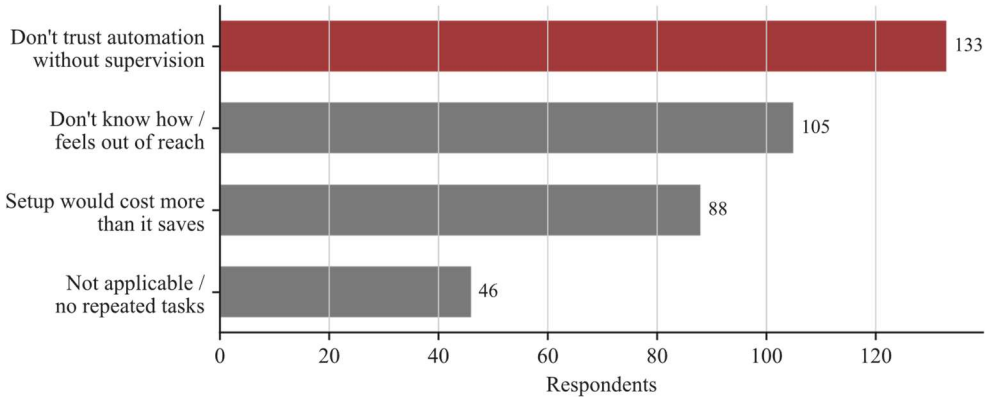


Figure 2.4: Reasons for not automating repeated manual tasks.

Distrust of automation and lack of know-how dominate.

The fifth is the access bargain that existing assistants impose, which is the sub-problem that distinguishes this project. A capable assistant is generally granted wide access to the machine, and the user is then unable to determine what it holds, what it may do, or how to reverse what it has done. Our questionnaire tested the alternative directly and found that 65 percent of respondents rate an assistant confined to explicitly added files as meaningfully more trustworthy than one with full access, with a mean of 3.8 on a five-point scale.

turnitin Page 30 of 78 - Integrity Submission Submission ID trn:oid:::3971:158456512

2.4 Goals and Objectives

The goal of the project is to demonstrate that a bounded, opt-in assistant can deliver useful multi-step automation while measurably improving user trust and preserving user control.

The objectives that realise this goal are as follows.

- To create a desktop helper that only works within folders that the user has granted access to, so that access beyond a grant is structurally impossible and not just forbidden.
- To minimize the time and effort users take to resume interrupted work and to carry out routine file, document and correspondence activities.
- To ensure that the user is in control of all the consequential actions by passing it through preview, approval and undo.
- To ensure that the system does not destroy any user data, by giving no code path to a destructive operation.
- To render visible and individually removable by a disclosure panel everything the system contains.
- To advocate voice as a full interaction mode that is subject to the same approval gates as typing.
- To test the system in terms of time to complete a task, cognitive workload, usability and trust, and to report inherited limitations in an honest manner.
- 2.5 Project Scope
- This project is to design, construct and test a desktop assistant to assist one user in managing daily work in clearly defined workspaces. The system will enable the user to:
- Assign a folder as a working directory, and unassign it to delete all the descendants.
- Resume interrupted work by looking at what was last active and what was to be done next.
- Clean up a folder by viewing and accepting a suggested set of moves.
- Synthesise notes in multiple documents with each claim traceable to its origin.
- Get an offer to automate a manual sequence once it has been repeated three times.
- Ask questions about source code and receive explanations and proposed fixes as differences
- Search the web on demand
- Draft electronic mail and send it only after approving the draft

- Always written as new files, draft documents.
- Perform any of the above by voice as well as by typing
- Check all that the system contains, and erase any of it.
- Undo a plan that has been approved as a unit.

The system will be implemented in Python. Multi-step plans will be produced using LangGraph [1], which provides an explicit pause point before any workspace-changing or outward call. Language model access will be local-first through Ollama [2], with an optional and per-workspace cloud fallback. Persistence will use local SQLite [3] together with a per-workspace vector index. The interface will comprise a command line surface and a local graphical control panel.

The project deliverables will be:

- A working desktop assistant implementing the features above
- This design and specification document, and the evaluation report that follows.
- A user manual
- A test suite of the boundary, policy, approval and undo guarantees.
- The project scope does not include the following:
- Any access to any part of the filesystem not within a granted workspace.
- Deletion or unsnapshotted overwriting of user data.
- Autonomous action, i.e. any file change or external action performed without authorization.
- Background browsing, screen recording, keystroke capture or always-on audio recording
- Mobile applications, in addition to the optional remote control surface in Chapter 6.
- Multi-user, shared or enterprise deployment
- Storing user information on any server of the project.

2.6 Sustainable Development Goal (SDG)

The project is aimed at Sustainable Development Goal 9, Industry, Innovation and Infrastructure, and specifically at 9.5, which is related to the improvement of scientific research and the modernization of technological capacities [4]. The complete list of goals is shown in Figure 2.5.

The local-first design of the project is its contribution to this goal. The default configuration of the system does its job with a model that runs on the personal machine of the user, so the functionality it offers is not contingent on constant connectivity to a data centre, on a subscription, or on whether the user is willing to send personal documents to a third party. This is important to the population that the

project is aimed at. Independent practitioners and students, especially in areas with patchy or expensive connectivity, are exactly the users where a cloud-dependent assistant is least available, and a local-first assistant brings an existing technological feature to them at a cost they can afford. Our data confirms the choice: local-first operation was the most popular deployment option among respondents at 40 percent, compared to 22 percent who actively favored cloud.

The project also supports Goal 8, Decent Work and Economic Growth, target 8.2 on productivity through technological upgrading, as the aim of the system is to eliminate repetitive friction in knowledge work and reallocate that time to productive work.

The goals are represented in the figure as numbered coloured tiles, each consisting of the goal number, its short title and a representative icon. The project is aimed at Goal 9, Industry, Innovation and Infrastructure, and secondarily Goal 8, Decent Work and Economic Growth.

The objectives that this project will respond to are presented clearly in Figure 2.6: SDG 9, Industry, Innovation an.d

Infrastructure (target 9.5) as the primary goal, and SDG 8, Decent Work and Economic Growth



Figure 2.5: The seventeen UN Sustainable Development Goals.

(target 8.2) as the secondary goal.



Figure 2.6: SDGs targeted by this project.

2.7 Constraints

The project operates under the following constraints, each of which shapes the design presented in Chapter 6.

- The system should be able to execute on a standard student laptop, which limits the size of the local language model, the resident memory footprint and the size of the acceptable index.
- The core operation should not need to be connected to the internet, and thus any feature that relies on the network should not fail but instead be limited to a specified threshold.
- The user data should not exit the machine unless there is an explicit per-workspace opt-in, which puts the egress decision in the model router instead of in any feature.
- The system should not have any inbound listening port outside the loopback interface, and thus the optional remote plane should not accept connections but should dial out.
- The credentials should be stored in the operating system keyring and should never be on disk in the system own storage tree.
- The team of three has to finish the project in two academic semesters, which means that the most risky parts should be constructed and tested as soon as possible.
- The analysis should be performed using the participants who are selected based on the networks of one institution, which limits the external validity of its findings.

2.8 Business Opportunity

The assistant market currently exists as a split between tools that can but require wide access and tools that can but require step-by-step guidance. The gap between them is the opportunity that this project will fill, and our data shows that the gap is not filled by a hypothetical preference: most respondents found a limited assistant more credible, and the most frequent reason they gave as to why they had not automated this work yet was distrust of automation.

Three conditions make the opportunity actionable. Local language models are now sufficiently powerful to execute useful inference on consumer hardware, and local-first operation is no longer aspirational but practical. The regulatory and personal sensitivity to data processing is on the increase, and it adds to the worth of an assistant that can say exactly what it contains. The target population is already familiar with assistants, since 77 percent of our respondents have used one, so the product does not have to establish the category.

The risk that corresponds is differentiation, and our data reveals it clearly. One of the common themes in free-text answers was that the idea seemed to be already addressed by the existing developer assistants. The solution is the bounded workspace and local-first operation, and the discovery is that this should be explicitly written in the product and not left to the user to deduce.

2.9 Stakeholders Description/ User Characteristics

The users of this system are those who do knowledge work on their own machines, along with the roles of evaluating and maintaining the project..

2.9.1 Stakeholders Summary

Table 2.1 identifies each stakeholder, their role with respect to the system, and their principal concern.

Table 2.1: Stakeholders and their concerns.

Stakeholder	Role	Primary concern
Student	Primary user. Works across documents, notes and source files on a personal laptop, frequently interrupted. Represents 81 percent of the research sample.	Recovering context quickly; keeping personal and academic files private; no configuration burden.
Developer	Primary user. Works across source files and documentation, and is the principal user of the code assistant. Represents 13 percent of the sample.	Correctness of proposed changes; never having code modified without review.
Freelancer or knowledge worker	Primary user. Manages client documents and correspondence.	Confidentiality of client material; reliable correspondence drafting with approval.

Project supervisor	Assesses the project's academic soundness and progress.	Traceability of design decisions to evidence; feasibility within the timeline.
Evaluation panel	Assesses the delivered project against its stated objectives.	Whether the hypothesis was tested and whether limitations are stated honestly.
Development team	Designs, implements and evaluates the system.	Preserving the architectural invariants under change; completing the scope in time.
Future maintainer	Extends the system after the project concludes.	Locating the enforcement points; understanding which properties must not be broken.

2.9.2 Key High-Level Goals and Problems of Stakeholders

15

The primary users share a single high-level goal, which is to spend less time on the mechanics of their work and more on its substance, and they share a single high-level problem, which is that the tools capable of granting this require a degree of access they are not prepared to give. Our research refines this general statement in four respects that bear directly on the design.

Users are divided on how much they wish to be asked. The questionnaire presented a choice between an assistant that seeks approval before every file change or outward action and one that acts on reasonable requests and permits review and undo afterwards. The split was 192 to 182, shown in Figure 2.7. No single interaction model satisfies this population, and approval granularity is therefore specified as a user-configurable setting in Chapter 4 rather than fixed in the design.

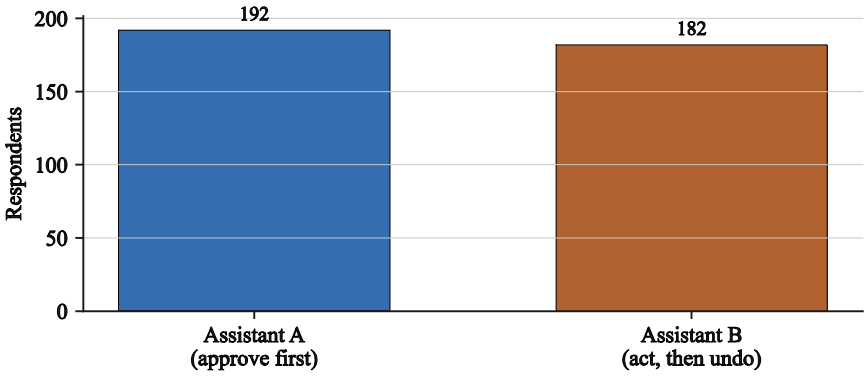


Figure 2.7: Approval-first versus act-then-undo.

The population is close to evenly divided.

Users will tolerate a mistake if they can reverse it. Asked what a single wrong or unwanted suggestion would make to their usage, 65 percent would still use the system, some more cautiously, and only 10 percent would give up. This observation confirms that a visible and reliable undo is more useful than an unattainable perfection, and it is why the undo journal is defined as a necessity and not a luxury.

Users will stop for reasons of fit rather than friction. The most common reasons that were predicted to abandon the system after a week were that it did not fit existing habits, at 27 percent, and that it made a mistake that the user could not trust it to avoid in the future, at 24 percent. Approval fatigue was the smallest named risk at 10 percent. This moderates an instinct of natural design: minimizing the number of prompts would be the least frequent worry, and would undermine the trust that the approval model offers.

Users desire the system to know their structure, and this is a request that the bounded design must consider keenly. The most requested feature that was not already planned was that the assistant should learn how the user organises their folders over time. This is defined as per-workspace memory, which can be seen and deleted in the disclosure panel, such that personalisation does not expand the bounded-scope guarantee silently.

2.10 Conclusion

The vision of the project has been mentioned in this chapter. It defined a two-half problem, the expense of restarting interrupted work and the reluctance of many users to grant an assistant wide access, and it framed the work by objectives, a scope, exclusions and constraints. It determined the sustainable development goal that the project is supporting, the opportunity that it is addressing and the stakeholders whose characteristics dictate the design. All this is supported by the underlying research, the distributions of which are provided in Appendix A. The next chapter is a review of the literature against which these claims are tested.

Chapter 3 Literature Review / Related Work.

This chapter is a review of the research literature and the applications that are available in the project. The review discusses 26 studies that are grouped into seven themes, each of which is a capability or a guarantee that the system should offer. For each theme the chapter summarises the work, analyses its strengths and weaknesses, and states the relationship to the proposed system. The chapter ends by determining where the project actually seals a gap, where it seals a gap in part, and where it inherits a limitation it cannot boast of having addressed.

3.1 Definitions, Acronyms, and Abbreviations.

The terms listed below are used in this chapter specifically, in addition to those defined in Chapter 1.

Agentic system: a system where a language model plans a sequence of steps and executes them by calling tools to execute them, instead of just generating text.

Human-in-the-loop: a system where a human being is required to approve or correct the action that a system has proposed before it can be implemented.

Prompt injection: an attack where instructions in retrieved content are interpreted by a model as user instructions.

Hallucination: content created that is not justified by the source material it claims to summarise.

Attribution: the fact that a statement generated is actually supported by the source it refers to.

Faithfulness: the similarity of a generated summary to the document being summarised.

Approval fatigue: the reduction in scrutiny that arises when a user is requested to approve actions is an architectural defence that isolates a trusted component against untrusted data or code.

3.2 Detailed Literature Review

11 The review is organised into seven themes. Themes one, two, and three are about the core behaviour of the system, which includes task resumption, limited agents and trust, and agentic planning. Themes four and five relate to two capabilities, voice interaction and code assistance. Themes six and seven are about the accuracy of generated text, including hallucination and citation accuracy.

3.2.1.1 Task Interruption and Resumption.

This problem is started with an observational finding. An office study of normal computer use has determined that when individuals are interrupted in a task, on-screen indicators like an open window
24 instead of minimised are what allow them to re-engage in the task later [5]. The experiment was carried out in actual working conditions, not in a laboratory, and it was terminated at diagnosis: no
38 system was suggested to the case where it is no longer convenient to leave a window open, e.g. after a restart or a few days break.

This diagnosis was later transformed into a tool. ScreenTrack captures a background time-lapse of the screen and lets a user scroll through it visually to find and reopen something they recall seeing, based on the idea that people remember sequences of visual scenes better than file names or timestamps [6]. In a controlled laboratory experiment it fared well with recent and visually distinct activity.

The weaknesses and strengths of this pair are complementary. The strength of the field study is the ecological validity and the weakness is that it did not generate any intervention. The strength of ScreenTrack is that it operationalises the finding, and its weaknesses were admitted frankly by its authors: retrieval is slower with a longer history, the method relies on tasks being visually distinguishable and thus is not well suited to textheavy work, and it was not tested in long-term multi-application use. Another apparent conflict is between a continuous screen recording and the privacy expectations of the users that this project is aimed at.

The connection to the proposed work is not an extension but a deliberate divergence. The resumption feature specified in this report does not record the screen at all. It works based on the file index and on notes that the user has written, matching on file name, recency and explicit text, and is therefore not dependent on whether two screens are similar to each other. Since a workspace is delimited by construction, the issue of searching a long history is also avoided, as there is only a short list of recent and relevant items to display. The truthful qualification is that the resumption benefit is a design claim and not a demonstrated claim because the system has not been field-validated in the same way as the original study.

3.2.2 Limited Artificial Intelligence Agents and Trust.

Another body of literature poses the question of how the autonomy of an agent can be limited to boundaries that a person can rely on. One is developer-facing. A trace-based assurance system records all actions of an agent as a structured trace with machine-checkable contracts attached to each step, such that a failure can be traced to a precise point and re-executed deterministically, mediating each action as allow, rewrite or block based on per-agent capability constraints [7].

A recent security survey looks at the same space through the lens of systems that require human sign-off [8]. It examines privilege separation as an architectural defence, and compares it to the separation that an operating system provides between a trusted kernel and untrusted programs, and extends this to the isolation of a planning component against untrusted external data. It then looks at the constraints of human-inthe-loop approval itself, and its conclusion is not favourable: frequent approval prompts cause decision fatigue that negates the safety benefit they are designed to offer, and most users do not have the background to evaluate what they are being asked to approve, so they either approve risky actions without actually evaluating them, or they reject harmless ones. This is an open problem that the survey does not provide a solution to.

The advantage of this body of work is that the machinery to bound an agent is already in place and is well known. Its drawback, as it currently stands, is that it is designed to support organisations that have

security personnel and infrastructure as opposed to an individual operating an assistant on their own computer. Another interesting feature of the assurance framework is that it lacks a way to reverse an action after it has been executed.

The connection to the proposed work is two-way. The workspace model of this project is a plain-language implementation of the same concepts: a folder that the user chooses instead of a sandbox that an administrator describes, and a readable preview instead of a machine-checked contract. Opposite the assurance framework the project contributes exactly what the assurance framework lacks, which is reversal after the fact, and this is one of the most direct correspondences in the review. Against the security survey the project must yield a point instead of asserting one. It does not address approval fatigue. It is based on the same mechanism that the survey finds unresolved, and there is nothing in the current design that differentiates between a routine approval and a risky one or minimizes prompts as trust is built. This is a limitation that is inherited by adopting this method in the first place, and the analysis should test it explicitly, not suppose that the constrained workspace will eliminate it.

3.2.3 Planning and Undo, Agentic Reasoning.

A large literature has shifted language models out of text generation and into goal-oriented reasoning, step planning and action. Reflexion enables an agent to assess its own past efforts, compose verbal reflections and store them as episodic memory to enhance subsequent efforts without retraining, enhance performance on decision-making, reasoning and coding tasks [9]. Toolformer is more focused, training a model to invoke an external tool like a calculator or a search engine in a more or less self-supervised way when and how [10]. Generative Agents builds upon the memory-and-reflection concept by showing that a population of agents can act in a coherent way across simulated days by storing, retrieving and reflecting on experience, as well as reporting failures such as false retrieval and falsified memory detail [11]. ReAct combines reasoning, acting and observing in one loop and demonstrates that the combination is more effective in performance and interpretability, and is susceptible to repetitive loops of action and ineffective recovery in case an action yields an unhelpful result [12]. Tree of Thoughts allows a model to explore many lines of reasoning and retrace unpromising ones, generating large payoffs on puzzle-like problems at a real computational cost [13].

The strength of this literature is that it establishes reasoning, tool use and reflective memory as a workable substrate for agent behaviour. The common weakness of all of them is the environment in which each system is tested: Reflexion on its own history of attempts, Toolformer on single tool calls it cannot easily compose, Generative Agents in a sandbox world, and ReAct and Tree of Thoughts on benchmark tasks. None of them has a confined workspace of a real user in the context of normal

interrupted work, and none of them needs the sort of approval that would make consequential actions reversible.

The proposed work lies in that gap. It takes the underlying combination of reasoning, tool use and memory but applies it to the actual files, notes and stated intentions of a user instead of the history of the agent or a simulated environment, and it encases the resulting multi-step plans in preview, approval and undo, which none of these systems offer. The failure modes this literature reports are also a direct warning. Artificial memory and recursive reasoning loops characterize exactly the fallacies that the resumption and synthesis capabilities should protect against, and they suggest that the number of unsupervised steps between user checkpoints should be small instead of believing that a longer autonomous chain is safe.

3.2.4 Voice Interaction Parity

This theme is whether individuals can perform practical computer tasks as well by voice as by typing, and how voice interaction is to be assessed. The state of the field is determined by two systematic reviews. The former performs a bibliometric review of user experience with voice interfaces and suggests a six-category classification of research, including voice-related variables such as gender, accent, personality and style [14]. The second compares voice assistant usability to the ISO 9241-11 model of effectiveness, efficiency and satisfaction [15].

There are three empirical studies that provide tangible design advice. One shows that speech has a realtime indication of user experience that post-hoc questionnaires fail to measure, and pause duration and disfluency rate are additional measures [16]. Another looks at verbal cues in waiting times, discovering that a long wait is best managed by an indicator that not only tells what is going on but also adds a touch of humour, but a short wait can be managed by an indicator alone and no more [17]. A third deals with the delay in response when talking to virtual agents in free-form conversation, where natural conversational fillers are better than artificial tones [18].

The advantage of this literature is that it provides usable thresholds and design guidelines, such as a foursecond latency target. Its limitation, as both reviews mention, is that voice usability is researched in bits and that conventional usability measures do not consider voice-related issues like trust and cognitive load.

The connection to the proposed work is two-fold. In design, the waiting-indicator and filler results dictate the behavior of the voice mode during the production of a plan, which should be a spoken explanatory indicator, not silence or a generic tone. To assess, the fact that the literature agrees that voice as a full interaction mode compared to practical multi-step tasks is under-researched is the

contribution: this project will compare voice to typing in terms of ease of use, task time, cognitive workload and trust. The speech-analysis method is a non-design feature that is an optional addition to that assessment.

3.2.5.1 Human Review of Code Assistance.

There are five studies that discuss the circumstances in which language model assistance with code is helpful. HULA illustrates a plan, generate and human-approve loop in a real deployment, reporting an 82 percent plan approval rate and discovering that the quality of output is strongly dependent on the quality of the input context [19]. HiLDe is more granular, driving generation token by token, and reports two warnings: regeneration may silently overwrite previous user edits, and showing too many options overwhelms users [20]. An analysis of suggestion timing looks at when an inline assistant ought to show a suggestion and cautions against maximising the acceptance rate at the cost of usefulness [21]. A taxonomy of expert steering strategies recognizes the actions through which an individual redirects a model, such as pointing out a particular error, explaining a requirement and re-orienting the strategy, and observing that certain strategies assume expert knowledge [22]. Lastly, a review of language models as code reviewers identifies systematic overcorrection, where models generate requirements and label correct code as incorrect, and that longer explanations are worse than shorter ones [23].

The advantage of this literature is that it confirms the human-approval loop as a successful interaction model and not a fallback. Its general infirmity, which the overcorrection study says is most acute, is that model judgement concerning code is helpful but not reliable in itself, and that explanations do not render it more reliable.

The relationship to the proposed work is direct. The code assistant mentioned in this report takes the form of HULA but is scaled down: it displays the source of the workspace, answers questions, describes errors and suggests a fix as a difference that the user can accept, and does not open pull requests or operate within a repository. The warnings of HiLDe are converted into two design principles, namely to introduce a single proposed change at a time and never to change code that the user has not accepted. Since the assistant is not invoked continuously but instead provides suggestions on demand, the suggestion-timing issue does not occur, which is a conscious benefit of the on-demand model. The overcorrection result is the immediate reason why a change should never be applied automatically, and the mitigation is the difference-plus-approval design, which alleviates but does not eliminate the underlying limitation.

3.2.6 Artificial Intelligence Summarisation Hallucination.

Four studies follow this issue up to diagnosis to meta-evaluation. The former proved that the content of hallucinations was present in all abstractive summarisation systems that were tested, and entailment between source and output was the correct check [24]. FactCC was then succeeded by a lightweight, trainable consistency checker, the nearest thing in the literature to a component that might be directly adopted [25]. FRANK then benchmarked the available factuality metrics and discovered that none of the metrics identifies all types of errors [26]. SummEdits generalized the question to frontier models and discovered that even the most robust models rate faithfulness poorly [27].

The merit of this literature is that its conclusion is consistent over a decade: hallucination is widespread, standard measures of overlap fail to identify it, no single measure is consistently reliable, and models are poor predictors of their own accuracy. Its engineering weakness is that it provides no full solution, just partial detectors and the suggestion to integrate them.

The connection to the proposed work should be expressed as a gap and not a solution. The synthesis feature works on a small, user-selected set of documents, a more limited and controlled task than open-domain summarisation and thus offers a smaller surface on which to hallucinate. It does not kill it. The existing design has no faithfulness check, and thus this is an open gap and not something that the bounded workspace solves. The specified direction is an entailment-style or FactCCstyle check post-generation, with the user kept in the loop, and the SummEdits finding rule eliminates the tempting shortcut of requesting the model to check its own citations.

3.2.7 Retrieval-Augmented Generation Citation Accuracy

Three articles discuss the question of whether generated text actually substantiates the sources it refers to. The former provides a practical definition of attribution, which is presented in the form of whether a statement passes the test that the statement is true according to the source cited [28]. ALCE subsequently benchmarked the capability of models to produce text with citations and incomplete citation was prevalent [29]. A review of implemented generative search engines revealed that approximately a quarter of citations failed and approximately half of sentences were not completely supported [30].

The merit of this literature is that it provides a standard and empirical evidence of the extent to which real systems are below it. Its drawback is that it offers a measuring device and not a policing tool.

The relationship to the proposed work is partial. The synthesis option eliminates the retrieval component of the problem by pre-selecting the documents by the user, and the citation map in Chapter 6 brings each chunk back to its file and line. What the existing design fails to do is to ensure

that a citation is provided to support the claim that is attached to it, or that all claims have one. The correct calibration in this case is the audit of deployed products: even mature products with specific citation engineering fail these tests at significant rates, and this is a reason to consider citation verification as a necessary task and not to believe that citing out of a small known set is safe by default.

3.2.8 Related Development Projects and Applications.

86 In addition to the research literature, there are three families of existing application that are relevant to the project. The inline coding assistants built into development environments offer continuous suggestion in a single editor, and their advantage is immediacy and their disadvantage is that they only deal with code and work with whatever the editor can access. Multi-step plans are run by agentic command-line assistants on the machine of a developer, and their power is real multi-step capability
29 and their weakness is the scope of access they need and the difficulty of undoing what they have done. General-purpose conversational assistants are generally capable but have no long-term, structured connection to the filesystem of the user, so they cannot resume work or act on files unless material is provided to them each time.

31 Our own study shows that this landscape is not just a question of technical comparison but of positioning. One of the common themes in free-text answers was that the proposed system seemed to be already addressed by the existing developer assistants, which confirms that differentiation should be made a point of argument. The distinguishing properties are the enforced workspace boundary, default local-first operation, the structural unavailability of destructive operations, and the disclosure panel that causes the trust claim to be falsifiable and not merely persuasive..

3.3 Literature Review Summary Table

Table 3.1 summarises the reviewed studies, identifying the theme each addresses, the contribution it makes, its relevance to this project and its limitations.

Table 3.1: Summary of the reviewed literature.

Study	Theme	Relevance to this project	Limitations
Iqbal and Horvitz [5]	Resumption	Establishes that resumption depends on persistent cues, motivating the session resume feature	Diagnosis only; proposes no system; cues fail after restart or a multi-day gap

•

Study	Theme	Relevance to this project	Limitations
Hu and Lee [6]	Resumption	Demonstrates a resumption tool, whose weaknesses this project avoids structurally	Slow retrieval over long histories; requires visually distinct tasks; privacy cost of screen recording
Paduraru et al. [7]	Bounded agents	Contract mediation and capability limits inform the policy engine	Built for organisations with security staff; no mechanism to undo an executed action
Kim et al. [8]	Bounded agents	Privilege separation informs the planner and dispatcher split; documents approval fatigue	Names approval fatigue as an open problem without offering a solution
Shinn et al. [9]	Agentic planning	Episodic written memory as a substrate for agent behaviour	Evaluated on the agent's own past attempts, not on real user work
Schick et al. [10]	Agentic planning	Self-supervised tool invocation informs the tool contract	Isolated tool calls that cannot readily be chained into plans
Park et al. [11]	Agentic planning	Memory, retrieval and reflection architecture informs the memory design	Sandbox world; reports fabricated memory and incorrect retrieval
Yao et al. [12]	Agentic planning	Interleaved reasoning and acting informs the bounded reflection loop	Repetitive action loops; poor recovery from unhelpful observations
Yao et al. [13]	Agentic planning	Search over reasoning paths informs the cycle cap on planning	Substantial computational cost; benchmark tasks only
Deshmukh and Chalmeta [14]	Voice	Classification of voice user experience research informs evaluation measures	Voice usability studied in fragments; no unified framework

Dutsinma et al. [15]	Voice	ISO 9241-11 framing of usability informs the evaluation instrument	Standard metrics omit trust and cognitive load

Study	Theme	Relevance to this project	Limitations
Ma et al. [16]	Voice	Speech as a real-time experience signal; optional evaluation enhancement	Requires turn-by-turn audio capture, which this project does not plan
Zhang et al. [17]	Voice	Waiting-indicator design for periods while a plan is produced	Findings specific to the wait durations tested
Wang et al. [18]	Voice	Natural fillers preferred over tones during response delay	Studied in free-form conversation rather than task execution
Takerngsaksiri et al. [19]	Code assistance	Validates the plan, generate and approve loop at 82 percent plan approval	Output quality heavily dependent on supplied context
Anaya Gonzalez et al. [20]	Code assistance	Cautions against silent overwriting and excessive alternatives	Token-level granularity finer than this project requires
Mozannar et al. [21]	Code assistance	Establishes that ondemand invocation avoids the suggestiontiming problem	Applies to inline assistants; acceptance rate is a misleading objective
Chidambaram et al. [22]	Code assistance	Steering taxonomy informs follow-up prompts when a fix is rejected	Some strategies presuppose expert knowledge
Jin and Chen [23]	Code assistance	Direct justification for never applying a code change automatically	Models overcorrect; longer explanations worsen reliability
Maynez et al. [24]	Hallucination	Entailment identified as the appropriate faithfulness check	Hallucination present in every system tested; no complete remedy

Study	Theme	Relevance to this project	Limitations
Laban et al. [27]	Hallucination	Rules out self-checking of citations by the model	Even frontier models judge faithfulness poorly
Rashkin et al. [28]	Citation	Supplies the attribution standard for synthesis output	A standard rather than an enforcement mechanism
Gao et al. [29]	Citation	Benchmarks citation generation; motivates completeness checks	Incomplete citation common even in purpose-built systems
Liu et al. [30]	Citation	Calibrates expectations against deployed products	A quarter of citations unsupported; half of sentences not fully supported
Kryscinski et al. [25]	Hallucination	FactCC is the most directly adoptable consistency checker	Not currently used by this project; a stated open gap
Pagnoni et al. [26]	Hallucination	Warns against relying on a single faithfulness metric	No single metric detects every error category

The table gives the theme, contribution, relevance to this project and stated limitations of each study.

3.4 Conclusion

The reviewed literature establishes three conclusions for this project.

The project addresses a real gap in four aspects. It uses multi-step agentic planning on a limited, user-selected desktop workspace instead of a simulated world, a single task or an open repository, a combination that the planning literature leaves open. It exposes agent memory and allows it to be edited by the user via the disclosure panel, where the reviewed systems execute memory as an opaque background process and report fabricated and mis-retrieved entries. It offers reversal following a previewed action, which the assurance framework does not have at all. It assesses voice as a holistic interaction mode compared to practical multi-step tasks, which both voice reviews recognize as under-explored.

The project partially addresses three other gaps. Its limited capability scoping is a plain-language implementation of the capability limits and privilege separation the security literature talks about, sacrificing rigour to usability. Its resumption feature operationalises an established finding without having replicated the field validation of that study. Its code assistance uses a proven human-approval loop but is still susceptible to the errors of the underlying model, which approval reduces but does not eliminate.

The project has three limitations that it does not address, and they are mentioned here instead of being postponed. It fails to address approval fatigue, which the security survey records as an open issue in precisely the mechanism that this design relies on, and the test must evaluate it across multiple sessions. It has no

faithfulness check on synthesised text, thus the narrower task minimises but does not remove the exposure the summarisation literature records. It does not confirm that their claims are supported by citations or that all claims have a citation, which the citation literature demonstrates to be unreliable even in deployed products of high quality.

These findings inform the requirements in Chapter 4 and the design in Chapter 6, and they establish the conditions under which the evaluation in the second phase of the project should report its findings.

Chapter 4 Software Requirement Specifications.

This chapter specifies the requirements for the system. It enumerates the features, specifies the functional requirements and their users, enumerates the quality attributes and non-functional requirements, documents the assumptions on which the specification is based, outlines the principal use cases, outlines the hardware and software requirements, describes the interface and the database design, and discusses the risks. Every requirement is formulated in a verifiable manner, such that a completed implementation can be verified against it instead of being evaluated informally.

The requirements are named with a prefix and a number. Functional requirements are denoted as FR-1 and non-functional requirements as NFR-1 and quality attributes as QA-1.

4.1 List of Features

The system offers the following features, in the sequence in which they are constructed. The order is based on measured demand, which is reported in Chapter 3, and technical risk, such that the elements on which all other features rely are built and tested first..

The workspace management, where a user gives a folder, checks what the system knows about it, and takes it away along with all that is based on it.

Action gating, where all proposed actions are categorized by risk, previewed, approved or rejected and reversed where necessary.

Session resume, which reinstates the user context following an interruption by reporting on what was last active and what was next to be done.

Folder tidy, which suggests a series of file moves, which the user previews and accepts as a single set.

Repeated task detection, which offers to automate a manual sequence after it has been observed three times

Synthesis of notes, where material is collected in multiple documents with each claim being citation to the source.

Code assistance, explaining errors and suggesting fixes as differences to be reviewed.

Web search, which is done at the request of the user.

Mail composition where a draft is written and dispatched after the user has approved it.

Production of a first draft as a new file, document drafting.

Speech input and output, which offers voice as an alternative interaction mode, which is also subject to the same approval gates.

- The disclosure panel, which reports everything the system currently holds and permits deletion of any part of it

The measured demand that determines this order is shown in Figure 4.1. Session resume, notes synthesis and folder tidy are the three most requested features, and speech is last by a wide margin, which is why it is built last.

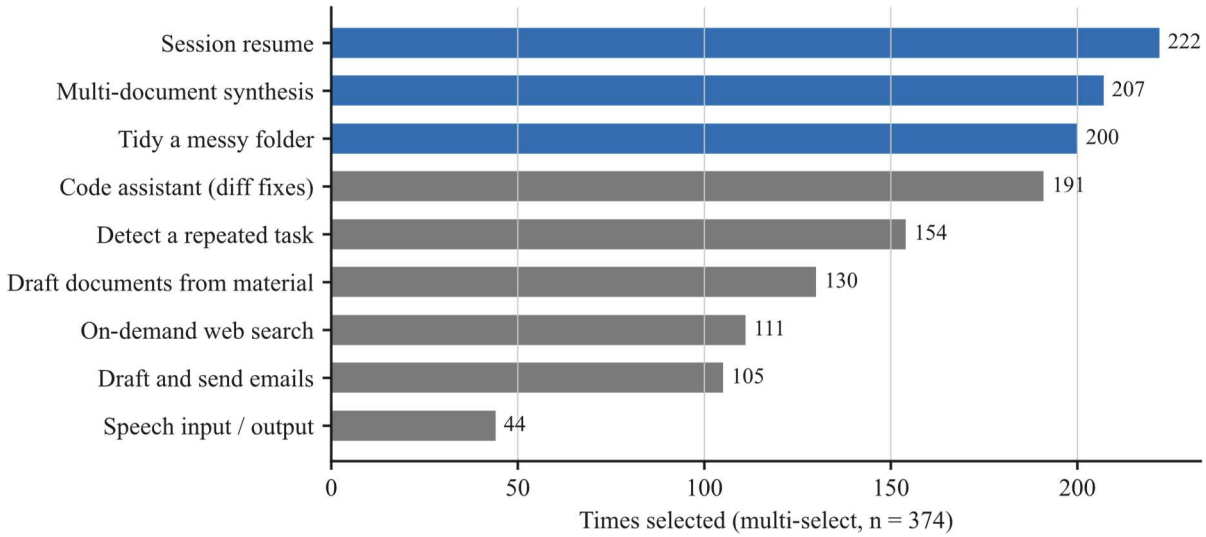


Figure 4.1: Feature demand (n = 374).

Demand is concentrated on session resume, notes synthesis and folder tidy.

4.2 Functional Requirements

6 The functional requirements describe the external behaviour of the system. Each is stated with the user who exercises it.

4.2.1 Workspace Management

- FR-1 The system will enable the user to make a folder a workspace, which will record its name, canonical root path, permission level and cloud policy. User: all primary users.
- FR-2 The system will not resolve any path the canonical form of which is not in the root of an active grant. User: all primary users, indirectly.
- FR-3 The system will canonicalise a path, including symbolic links and relative segments, and then test whether it is contained in a grant. User: all primary users, indirectly.
-
- FR-4 The system will not resolve paths that match a denylist of credential and system locations, even when the path is in a granted folder. User: all primary users, indirectly.
- FR-5 The system will enable the user to cancel a workspace, which will remove the index entries, embeddings, sessions, memory and rules, but not the files of the user. User: all primary users.

-
- FR-6 The system will have an index of the files in each granted workspace, which will be updated incrementally as files change. User: all primary users, indirectly.
- 4.2.2 Action Gating and Recovery
- FR-7 The system will categorize all the proposed actions into precisely one of four risk levels, which are read, reversible, outward and destructive. User: all primary users, indirectly.
- FR-8 The system will perform read operations in a granted workspace without authorization. User: all primary users.
- FR-9 The system will store all reversible actions pending approval, unless the user has already decided to allow that operation in that workspace without prompting. User: all primary users.
- FR-10 The system will retain all outward actions pending approval in any situation, and will not allow this to be set off. User: all primary users.
- FR-11 The system will reject all destructive actions, and it will not give any approval route through which it can be done. User: all primary users.
- FR-12 The system will display, prior to any gated action, a preview of the workspace being impacted, the operations being proposed, the risk level, whether anything is leaving the machine, and whether anything is being deleted. User: all primary users.
- FR-13 The system will enable the user to approve, approve and remember, edit or reject a pending plan. User: all primary users.
- FR-14 The system will scope a remembered approval to a single operation in a single workspace, and will never remember an outward or destructive operation. User: all primary users.
- FR-15 The system will document the reversal of each change prior to the change. User: all primary users, indirectly.
- FR-16 The system will enable the user to undo an approved plan as a unit, and the steps will be restored in the reverse order. User: all primary users.
- FR-17 The system will not automatically accept a pending action, by default or under load, by default or by load. User: all primary users, indirectly.
- FR-18 The system will maintain a record of all proposals, refusals, approvals, rejections and executions in an append-only, hash-chained audit log. User: all primary users; auditor.
- 4.2.3 Capability Features

- FR-19 The system will provide, on demand, the files that have been recently used in a workspace and the last action that the user took, to allow the work to be restarted. User: all primary users.
- FR-20 The system will suggest a sorting of files in a folder, and all the moves will be previewed before any is executed. User: all primary users.
- FR-21 The system will identify a manual sequence that is repeated three or more times and will suggest to automate it, and will not install an automation without the user consent. User: all primary users.
- FR-22 The system will synthesise content across documents in a workspace, and each claim will be attached with a citation of its source file and location. User: all primary users.
- FR-23 The system will make unsupported any claim in a synthesis which cannot be traced to a source. User: all primary users.
- FR-24 The system will respond to questions regarding source code in a workspace and will display any proposed change as a difference, and will never write or execute code independently. User: developer.
- FR-25 The system will not search the web automatically, but only when an explicit user request is made, and will not search in the background. User: all primary users.
- FR-26 The system will prepare electronic mail in the form of a draft and will not be sent until the user has approved the draft. User: all primary users.
- FR-27 The system will create a written document as a new file and will never overwrite an existing one. User: all primary users.
- FR-28 The system will be able to receive spoken input and give spoken output, with the same approval gates as typed interaction. User: all primary users.
- FR-29 The system will only record audio when a voice session has been armed by the user. User: all primary users.
- 4.2.4 Disclosure and Model Access.
- FR-30 The system will provide a panel that reports all workspaces granted, files indexed, chunks embedded, facts remembered, sessions retained, automation rule, policy exception, action taken and item of data sent outside the machine. User: all primary users.
- FR-31 The system will enable the user to remove any item displayed in that panel except the audit log. User: all primary users.

- FR-32 The default configuration of the system will direct model requests to a local model. User: all primary users, indirectly.
- FR-33 The system will not send workspace content to a model tier that has been disabled by the cloud policy of the workspace. User: all primary users, indirectly.
- FR-34 The system will not retrieve any material not within the active grant, and will document each such exclusion. User: all primary users, indirectly.
- FR-35 The system will generate an inspectable plan without executing it, and will only allow execution by the dispatcher and only where a corresponding policy verdict exists. User: all primary users, indirectly.
- 4.3 Quality Attributes
- The quality attributes below declare the properties that the system should possess, along with the method of measuring each of them.
- QA-1 Confinement. Any operation that solves a path not within an active grant is an error. Evaluated by tests that seek traversal with relative segments, symbolic links to external the grant, absolute paths, and sibling directories with a common prefix in their name, all of which should be denied and recorded.
- QA-2 Non-destructiveness. There is no execution path that removes or overwrites user data without a snapshot. Evaluated by examining the operations registered by the dispatcher and by tests that destructive operations are denied instead of gated.
- QA-3 Recoverability. Any approved reversible plan is reversible. Evaluated by tests that run a plan, run it in reverse, and compare the state of the filesystem after running the plan to the state before running the plan.
- QA-4 Transparency. All the system contains is reportable and, except the audit log, deletable. Evaluated by comparing the counts reported by the panel with direct queries of each store..

- QA-5 Auditability. The history of activities cannot be changed without being noticed. Evaluated by re-computing the hash chain and ensuring that any alteration or deletion of an entry destroys it.
- QA-6 Responsiveness. The interactive path does not block on background work. Assessed by measuring time to first token while an initial workspace ingestion is in progress.
- QA-7 Degradability. The loss of any external capability results in a declared constraint and not a failure. Compared to the degradation contract in Chapter 6.
- QA-8 Modality parity. Voice interaction uses the same gates as typed interaction. Evaluated by ensuring that the voice mode introduction did not need any extra approval gate.
- 4.4 Non-Functional Requirements
- NFR-1 Performance. The interactive path will generate a first token in less than one second with a local model and in less than two seconds with a cloud model, on the reference hardware defined in Section 4.7.
- NFR-2 Performance. Incremental index updates will be debounced and will not block interactive requests. Modifying a single line of a file will result in the re-embedding of only the modified chunk.
- NFR-3 Capacity. The system will run under adjustable limits on memory, index size and processor share, and will be usable on a consumer laptop during indexing.
- NFR-4 Concurrency. The system has a single interactive user per host instance. Background work will be implemented using a bounded priority queue that will drop the lowest-priority work when full instead of expanding indefinitely.
- NFR-5 Availability. The basic functionality will be maintained without the internet. Actions that need the network will fail as explicit, retriable actions instead of being silently dropped.
- NFR-6 Privacy. User documents, embeddings, session history, memory, credentials and audit logs will not be sent to any server operated by the project.
- NFR-7 Security. The credentials will be stored in the operating system keyring and will never be written to the system own storage tree. At rest, data will be encrypted.
- NFR-8 Security. The system will not open an inbound listening port except on the loopback interface.

- NFR-9 Usability. The granularity of approval will be user-configurable, allowing approval of all actions, approval by category, or session-based trust following initial approvals.
- NFR-10 Usability. Background work will report visible, cancellable progress instead of an indeterminate indicator.
- NFR-11 Reusability. Capability features will be provided as a uniform tool contract, with a new feature being added without changing the gateway.
- NFR-12 Extensibility. Model providers will be configuration, not code branches, such that any compatible endpoint can be registered without changing the router.
- NFR-13 Portability. The system will be based on Windows, macOS and Linux, and will not rely on any platform-specific filesystem behaviour to provide its confinement guarantee.
- NFR-14 Maintainability. Path resolution will be in one component and execution in one other, such that every guarantee has a single enforcement point.

4.5 Assumptions

- The specification is based on the following assumptions.
- The user is an individual who has his/her own machine and has the right to access the folders of his/her choice.
- The host machine has enough resources to execute a small local language model, or the user has configured cloud fallback on the workspaces in question.
- The operating system offers a keyring that is appropriate to store credentials and a canonicalisation algorithm that correctly resolves symbolic links.
- The metadata that incremental indexing relies on, i.e. modification times and stable path identity, are supported by the filesystem.
- Workspaces that are granted as folders are not concurrently modified by some other automated process in a way that would invalidate a snapshot between its capture and the next operation.
- The user can read a preview of what will happen and make a decision about it, which the literature indicates is an assumption and not a guarantee and which the assessment should investigate.
- The documents in a workspace can be adversarial, as a file can be created by any source, and the design thus considers retrieved content as data, not as instruction.

4.6 Use Cases

This section presents the use cases that exercise the central functionality of the system or that illustrate a delicate point of the architecture. Four are given: granting a workspace, which establishes the boundary; tidying a folder, which exercises the gating and recovery path end to end; attempting to reach outside a workspace, which demonstrates the confinement guarantee; and sending mail, which demonstrates that an outward action cannot be configured past its gate.

Table 4.1: Use case UC-1: grant a folder as a workspace.

Name	UC-1 Grant a folder as a workspace
Actors	Student, Developer, Freelancer
Summary	The user selects a folder and opts it in as a workspace. The system records the grant, indexes the folder, and reports what it now holds.
Pre-Conditions	The folder exists and is readable by the user. The folder is not already granted.
Post-Conditions	A grant exists with a canonical root, a permission level and a cloud policy. The folder's files are indexed and the grant is recorded in the audit log.
Special Requirements	The root is stored canonically, so that a later symbolic link cannot alter the region the grant covers.

Basic Flow			
Actor Action		System Response	
1	The user selects a folder and names it.	2	The system canonicalises the path and records the grant with its permission and cloud policy.
3	The user confirms the permission level.	4	The system indexes the folder in the background, reporting visible progress, and writes the grant to the audit log.
Alternative Flow			
1-A	The user selects a folder already granted.	2-A	The system reports that the folder is already a workspace and makes no change.

Table 4.2: Use case UC-2: tidy a folder.

Name	UC-2 Tidy a folder
Actors	Student, Developer, Freelancer

Table 4.2: continued from previous page

Summary	The user asks the system to tidy a folder. The system proposes a plan of moves, the user previews and approves it, and the system executes it with undo available afterwards.
Pre-Conditions	The folder lies within a granted workspace with write permission. The workspace has been indexed.
Post-Conditions	The approved moves have been performed, the inverse of each is recorded, and the plan may be reversed as one unit. Nothing has been deleted.
Special Requirements	The snapshot is recorded before any move is performed. Undo operates at the granularity of the plan, not of the individual move.

Basic Flow			
Actor Action		System Response	
1	The user asks the system to tidy a folder.	2	The system lists the folder, reads metadata, and produces a plan of proposed moves.
		3	The system classifies each step, and presents a preview stating the risk tier, that nothing leaves the machine and that nothing is deleted.
4	The user reviews the moves and approves the plan.	5	The system records the inverse of each move, performs the moves, writes the audit entries and reports that undo is available.
Alternative Flow			
4-A	The user rejects the plan.	5-A	The system performs nothing, records the rejection in the audit log and halts cleanly.
4-B	The user approves, then later requests undo.	5-B	The system reverses every move in the plan in reverse order and reports any step it could not reverse.

Table 4.3: Use case UC-3: attempt to reach outside a workspace.

Name	UC-3 Attempt to reach outside a workspace
Actors	Capability service; indirectly, an adversarial document

Summary	A proposed action references a path outside the grant, whether through a relative segment, a symbolic link or an absolute path. The system refuses it and records the attempt.

Table 4.3: continued from previous page

Pre-Conditions	A workspace is granted and active.
Post-Conditions	No filesystem access occurred outside the grant. The refusal is recorded in the audit log with its reason.
Special Requirements	Canonicalisation precedes the containment test, so that a path which merely appears contained is resolved to its true location first.

Basic Flow			
Actor Action		System Response	
1	A service requests a path containing a traversal segment.	2	The broker canonicalises the path and finds it outside the grant root.
		3	The broker records the refusal and raises it, so no access occurs.
Alternative Flow			
1-A	The path is a symbolic link whose target lies outside the grant.	2-A	Canonicalisation resolves the link before the containment test, so the refusal is identical.
1-B	The path names a credential file inside the grant.	2-B	The deny list refuses it even though it lies within the granted folder.

Table 4.4: Use case UC-4: draft and send electronic mail.

Name	UC-4 Draft and send electronic mail
Actors	Freelancer, Student, Developer
Summary	The user asks the system to prepare a message. The system drafts it and holds it until the user approves the exact content and recipient.
Pre-Conditions	A mail account is configured and its credentials are held in the keyring. Any material referenced lies within a granted workspace.

Post-Conditions	The message is sent only if approved. The egress is recorded in the audit log with its recipient and time.
Special Requirements	This gate is not configurable. A remembered approval cannot be recorded for an outward operation.
Basic Flow	

Table 4.4: continued from previous page

Actor Action		System Response	
1	The user asks for a message to be prepared.	2	The system drafts the message and classifies the send step as an outward action.
		3	The system presents the recipient, subject and full body for approval.
4	The user reads the draft and approves it.	5	The system sends the message and records the egress in the audit log.
Alternative Flow			
4-A	The user requests that the approval be remembered.	5-A	The system declines to record the rule, because outward operations are never remembered.
4-B	The user rejects the draft.	5-B	Nothing is sent and the rejection is recorded.

The four use cases together exercise the principal architectural elements. UC-1 establishes the boundary and the index. UC-2 traverses classification, preview, approval, snapshot, execution, audit and reversal, which is the full length of the request lifecycle. UC-3 demonstrates the confinement guarantee against the three forms of escape a bounded system must resist. UC-4 demonstrates that the outward gate is a property of the architecture rather than a configurable preference.

4.7 Hardware and Software Requirements

4.7.1 Hardware Requirements

Table 4.5 states the minimum and recommended hardware for the host machine. The minimum configuration is intended to run the system with a small local model, and the recommended configuration is the reference against which the performance requirements in Section 4.4 are measured.

Table 4.5: Host hardware requirements.

Component	Minimum	Recommended
Processor	Quad-core, 2.0 GHz	Eight-core, 2.5 GHz or higher
Memory	8 GB	16 GB or more, permitting a larger local model to remain resident
Storage	10 GB free, solid-state	25 GB free, solid-state, for indexes and snapshots

Table 4.5: continued from previous page

Component	Minimum	Recommended
Graphics	Not required; inference on processor	Dedicated graphics with 6 GB memory or more, for local inference
Audio	Microphone and output device, for the voice mode	As minimum
Network	Not required for core operation	Broadband, for cloud fallback and web search

4.7.2 Software Requirements

Table 4.6 states the software required to develop and operate the system, together with the purpose each serves.

Table 4.6: Software requirements.

Component	Version	Purpose
Operating system	Windows 10 or later, macOS 12 or later, or a current Linux distribution	Host platform
Python	3.11 or later	Implementation language for the entire system
LangGraph [1]	Current	Multi-step planning with an explicit pause before consequential calls
Ollama [2]	Current	Local model runtime, the default inference tier

SQLite [3]	3.35 or later	Local system of record for metadata, sessions, rules and audit
Local vector index	Current	Per-workspace embedding storage and retrieval
Typer	0.12 or later	Command line interface
Gradio	6.0 or later	Local graphical control panel served on the loopback interface

Table 4.6: continued from previous page

Component	Version	Purpose
Watchdog	4.0 or later	Filesystem change detection for incremental indexing
Operating system keyring	Platform-provided	Credential storage outside the system's own tree
Pytest	8.0 or later	Automated verification of the boundary, policy, approval and undo guarantees

4.8 Graphical User Interface

The system has three surfaces sharing a single view model, such that the same state is seen no matter how it is arrived at. The command line surface is scriptable and the surface on which configuration and diagnosis is done. The main surface is the local graphical control panel, which is only served on the loopback interface. The optional remote progressive web application, as outlined in Chapter 6, displays the same view model on a different device and does not have any extra privilege.

The control panel has five views. The chat view takes a typed or spoken request and shows the response along with any plan generated. The approval view shows a pending plan with its preview, the workspace to be affected, the operations to be performed, the risk level, anything leaves the machine or anything is deleted, and approve, approve and remember, edit and reject. The disclosure view enforces FR-30 and FR-31, enumerating all types of held data and their number and allowing deletion. FR-1 and FR-5 are implemented by the workspace view, which lists grants and provides revocation. The activity view presents the audit log and the undo history, implementing FR-16 and FR-18.

Navigation is a request lifecycle. A user starts in the chat view, is redirected to the approval view when a plan needs a decision, and is redirected to the chat view with the result and an undo affordance. The

disclosure, workspace and activity views can be accessed at any time and are not included in the request path. The implementation is annotated with each view and the requirements it fulfills, so that the interface can be checked against the specification instead of being evaluated by impression.

The flow of navigation is presented in Figure 4.2, and the screens of the main surface are presented in Figures 4.3 to 4.10. The request path (FR-8 to FR-16 and FR-28) is covered by the Work screen, the Folders screen covers FR-1 and FR-5, and the What it knows screen covers FR-30 and FR-31. The model, mail and remote access options, and the interface can be changed to a dark theme in the Settings screen.e.

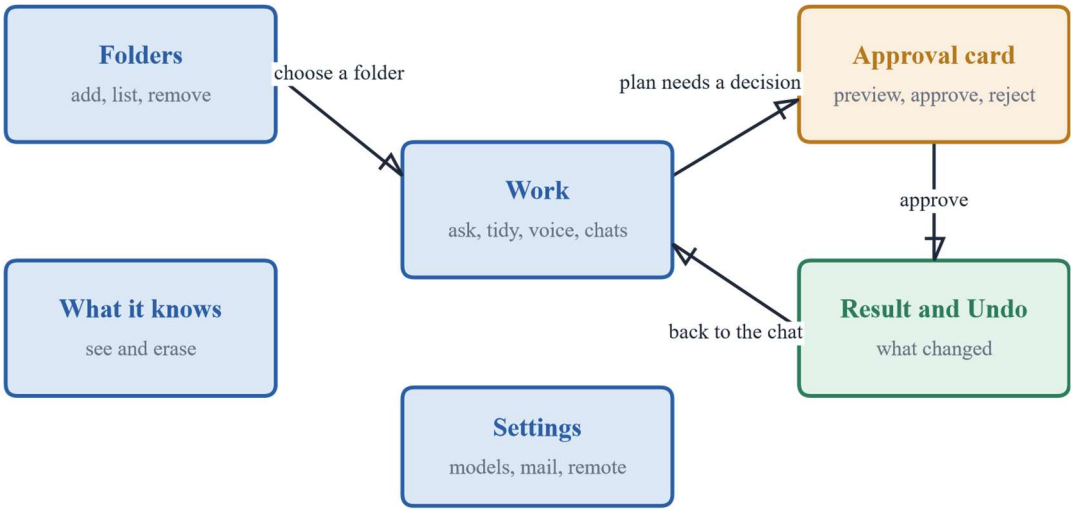


Figure 4.2: Navigation flow of the control panel.

The Work screen is where a request begins. The user chooses a folder, types or speaks what is wanted and presses Ask, or uses one of the shortcut buttons for a tidy-up, for what changed and for repeated tasks. The voice controls sit in a collapsible panel, and the saved chats are listed in the sidebar.

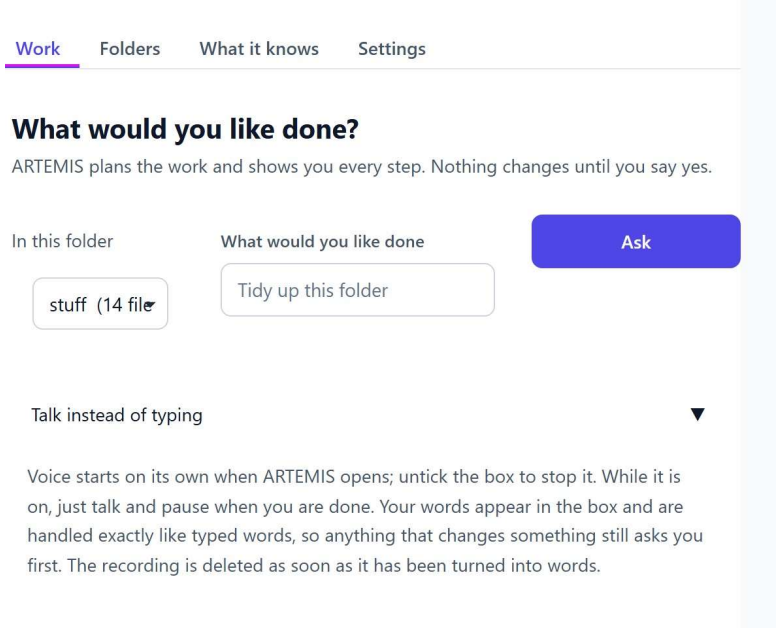


Figure 4.3: Work screen: ask, tidy and voice.

The Folders screen lets the user opt a folder in by typing its path or browsing to it. Each granted folder is listed with the number of files it holds, the permission it carries and whether cloud models are allowed for it. A folder that has not been added here cannot be seen by the assistant at all.

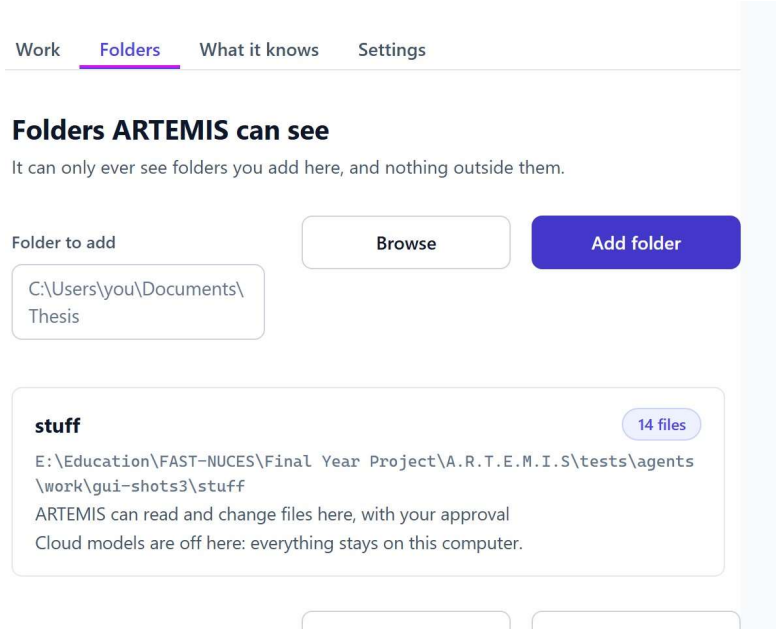


Figure 4.4: Folders screen: add and list folders.

The What it knows screen implements the disclosure requirement. For the chosen folder it lists every file the assistant has indexed, offers a check for changes, and offers a button that forgets the folder. Forgetting removes what the assistant holds and never touches the user's own files.

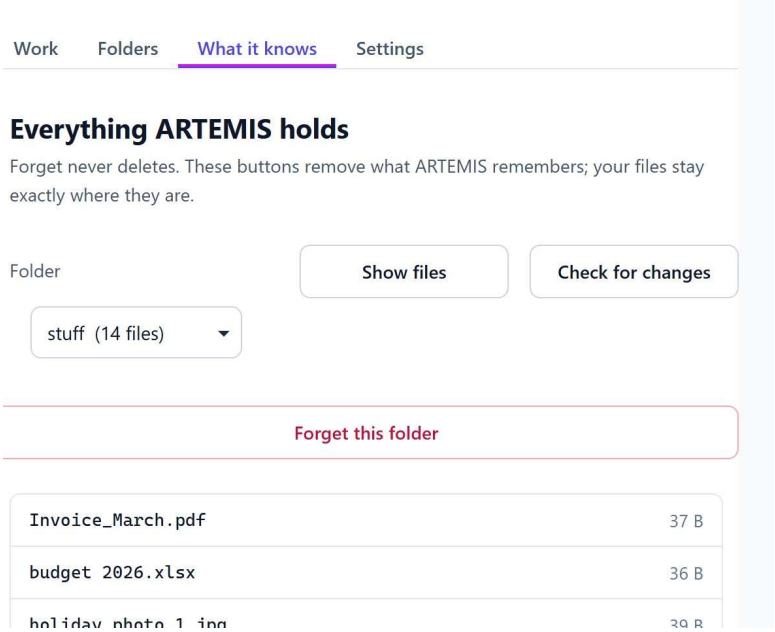


Figure 4.5: What it knows screen: see and erase.

When a request would change files, the assistant does not act. It shows a plan in plain words and three choices: Approve, Approve and do not ask again for this kind of change, or No, leave it. Nothing has happened until the user presses one of them.

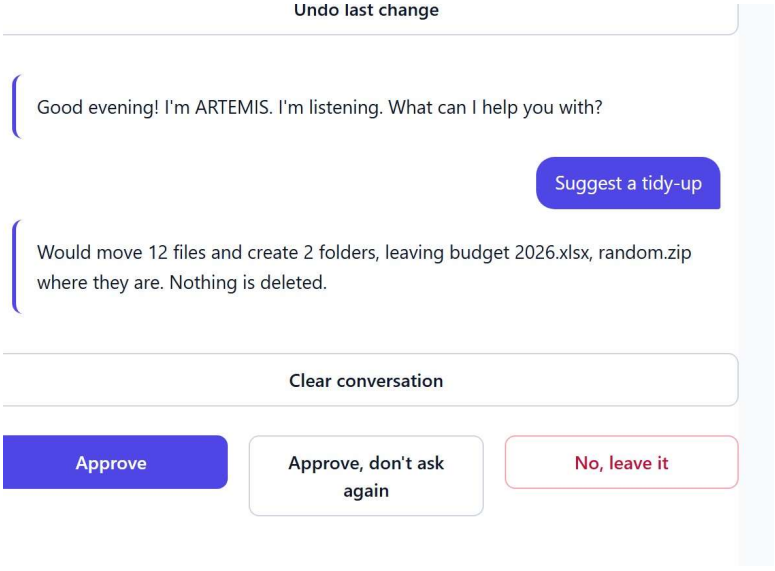


Figure 4.6: Approval card for a tidy-up plan.

After approval the assistant reports what it did, and the Undo last change button reverses the whole plan as one unit. Here a tidy-up that created two folders and moved twelve files is undone in a single step.

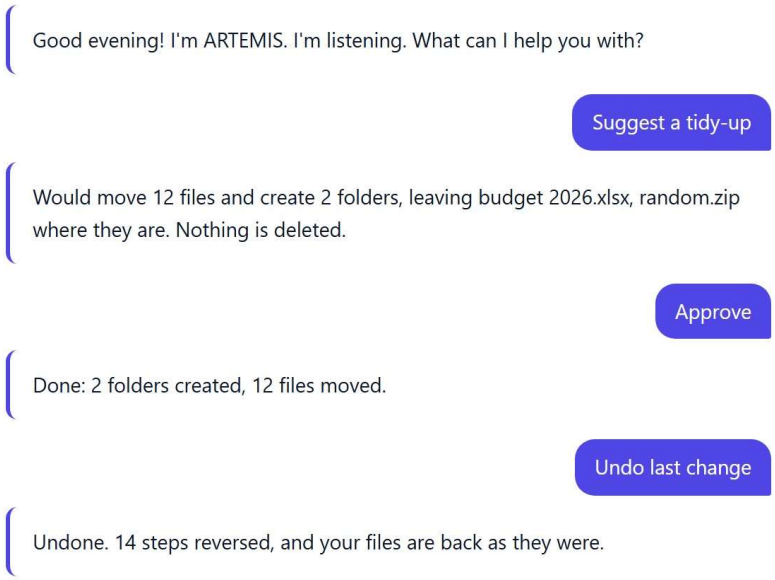


Figure 4.7: A tidy-up, its approval and its undo.

The Settings screen holds the choices that are not part of everyday use: where the assistant keeps what it knows, which models it may use, the mail account, and the remote access controls.

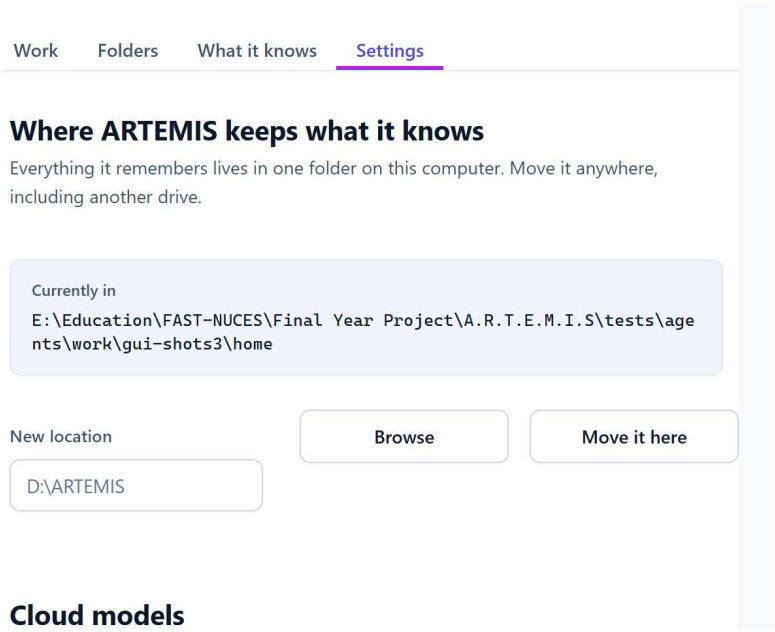


Figure 4.8: Settings screen: data location and cloud models.

Remote access is optional and off until the user connects to a relay. The screen states that messages are scrambled on the phone and only unscrambled on the computer, and it lists the folders a paired device may use, none by default.

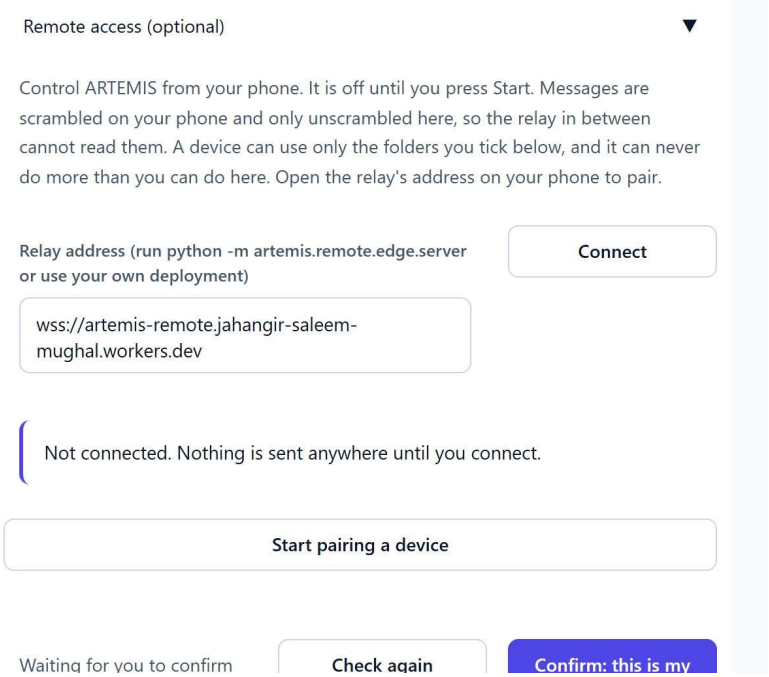


Figure 4.9: Remote access controls in Settings.

The interface can be switched between a light and a dark theme with one button, and the same screens and controls are shown in both.

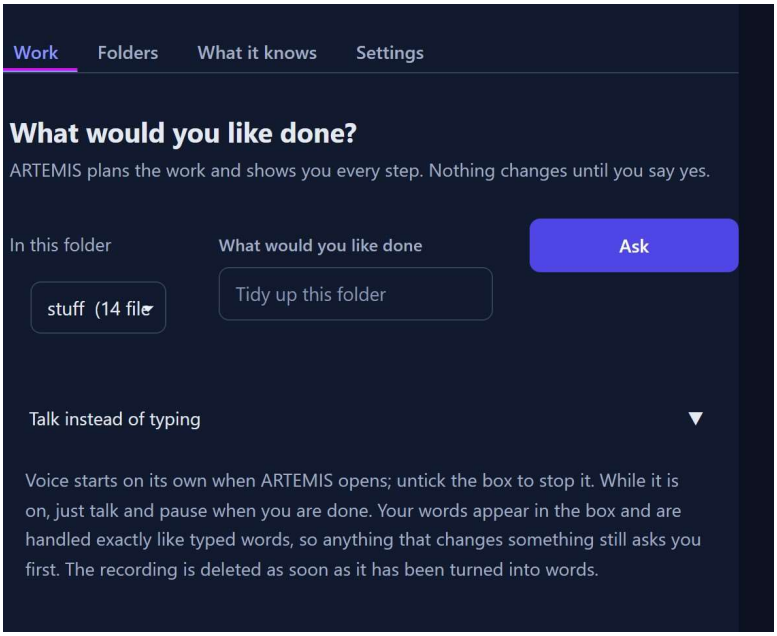


Figure 4.10: The Work screen in the dark theme.

4.9 Database Design

The system is based on SQLite as its system of record and a per-workspace vector index stored on the filesystem. The relational store is small and highly normalised, and all tables except the workspace table are subordinate to a grant, which is what allows the expression of revocation as a single cascading deletion.

4.9.1 ER Diagram

The workspace entity is the centre of the entity relationship structure. There are numerous indexed files, numerous sessions, numerous memory entries, numerous automation rules, numerous policy rules and numerous undo journal entries and all these relationships are one-to-many identifying relationships with deletion cascading down to the workspace. A session consists of numerous turns, once more cascading. The audit log is explicitly external to this structure: it is a reference to a workspace, but not owned by a workspace, since an audit record of a revoked workspace must outlive that revocation, and a log of its subject can be erased is not an audit log.

This structure articulates two demands. FR-5 is made the cascade, such that the revocation of a grant automatically removes all that is based on it instead of depending on a cleanup routine that may not be complete. FR-18 is the lack of a cascade on the audit log, such that the history of what has happened cannot be erased by deleting its object. The resulting structure is illustrated in figure 4.11..

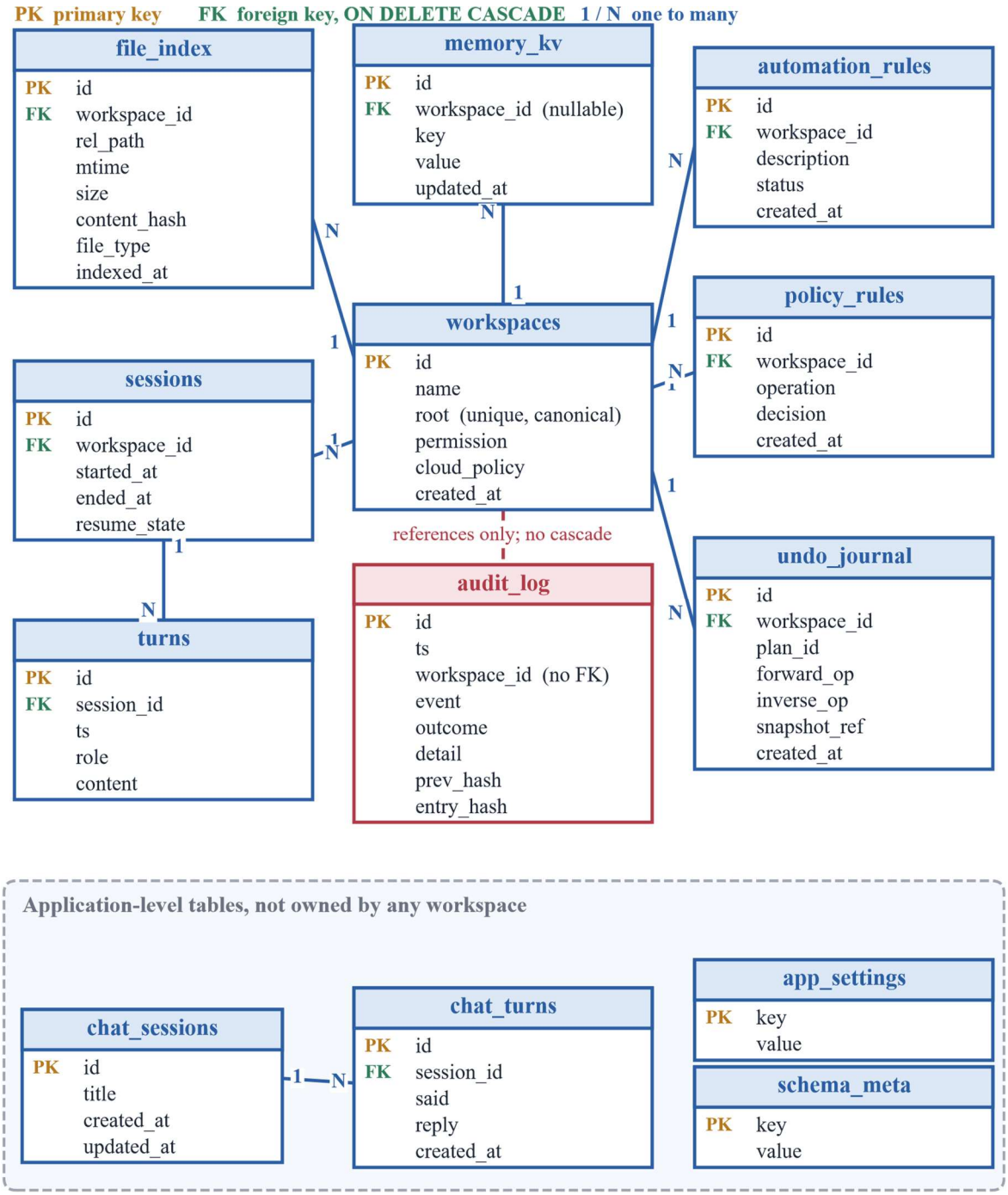


Figure 4.11: Entity relationship diagram.

4.9.2 Data Dictionary

Table 4.7 defines each table, its purpose and its principal attributes.

Table 4.7: Data dictionary of the relational system of record.

Table	Purpose	Principal attributes
workspaces	One row per folder the user has granted	id; name; root, stored canonically and unique; permission, being read only or read and write; cloud policy; created at

file index	What exists and where, mechanical and rebuildable	id; workspace id; relative path; modification time; size; content hash; file type; indexed at. Unique on workspace and relative path
audit log	Append-only, hash-chained record of proposals, refusals and actions	id; timestamp; workspace id; event; outcome; detail; previous hash; entry hash
sessions	Session lifecycle and resume state	id; workspace id; started at; ended at; resume state
turns	Conversation history within a session	id; session id; timestamp; role; content
memory _kv	Interpretive long-term memory: preferences, facts, named entities	id; workspace id; key; value; updated at. Unique on workspace and key
automation rules	Proposals and accepted rules from repeated task detection	id; workspace id; description; status, being proposed or accepted; created at
policy rules	User-configured approval preferences, scoped to one operation in one workspace	id; workspace id; operation; decision; created at
undo journal	Forward and inverse operations with snapshot references	id; workspace id; plan id; forward operation; inverse operation; snapshot reference; created at

The trust claim and merit explicit statement are carried by two design points in this schema. Referential integrity should be turned on all connections, as the database engine defaults it off and the cascade clauses would otherwise be silently ineffective, which would result in revocation leaving data behind and seemingly succeed. The hash chain of the audit log connects one entry to the previous one, such that any attempt to modify or delete an entry disrupts the chain and the integrity of the record can be checked instead of being presumed.

In addition to the relational store, every workspace has its own vector index in a different directory. This seclusion is intentional. It implies that the embeddings of a workspace cannot be accessed when another workspace is open even in the case of a query defect, as the index is not open, and it makes workspace revocation a deletion of a directory. Snapshots of content are stored in a common blob store, such that duplicate snapshots of a file that has not changed are free.

4.10 Risk Analysis

36 Table 4.8 lists the major risks to the project, evaluating each in terms of likelihood and impact and indicating its mitigation.n.

Table 4.8: Project risks.

ID	Risk	Likelihood	Impact	Mitigation
R-1	A local model small enough for a student laptop proves inadequate for the planning tasks	Medium	High	Bounded failover to a cloud tier under per-workspace opt-in; capability probing in the router; degradation to a stated limitation rather than an error
R-2	Approval fatigue erodes the safety benefit the gates provide	High	Medium	Configurable granularity and scoped remembered approvals; outward gates remain fixed; the evaluation tests for fatigue directly rather than assuming its absence
R-3	A confinement defect permits access outside a grant	Low	Critical	Resolution confined to one component; canonicalisation before containment; adversarial tests for traversal, symbolic links, absolute paths and name-prefix siblings
R-4	Synthesised text contains unsupported claims	High	Medium	Citation map carried through to the answer; unsupported claims rendered as such; a faithfulness check is identified as an open gap rather than claimed as solved
R-5	Prompt injection from an adversarial document causes an unintended action	Medium	High	Provenance tagging; a structured plan as the interface; policy evaluated out of band; user preview of outward actions; blast radius bounded by the workspace

Table 4.8: continued from previous page

ID	Risk	Likelihood	Impact	Mitigation

R-6	Index growth degrades retrieval or rebuild time	Medium	Medium	Per-workspace indexes; incremental re-embedding by content hash; two-stage retrieval; configurable ceilings
R-7	The scope proves too large for three members across two semesters	Medium	High	Staged build order with the boundary and gates first; the remote plane is optional by construction and may be dropped without affecting the rest
R-8	The audience perceives no differentiation from existing assistants	Medium	Medium	Bounded workspace and local-first operation stated explicitly in the product; the disclosure panel makes the claim falsifiable
R-9	Evaluation participants are drawn from one institution, limiting generalisability	High	Low	Stated as a limitation; segment analysis reported; conclusions framed to the studied population
R-10	Undo fails to restore state after a partially completed plan	Low	High	Snapshots recorded before mutation; reversal in reverse order; irreversible steps reported rather than silently skipped

4.11 Conclusion

This chapter has defined what the system should do and in what circumstances. It enumerated the features and 35 functional requirements, each of which is formulated in a manner that it can be verified, the quality attributes and nonfunctional requirements, the assumptions, four use cases that test the key architectural components, the hardware and software requirements, the interface, the database design and the major risks. The specification provides the verification criteria that Chapter 7 applies to its test cases and Chapter 9 applies to the prototype. The next chapter describes the approach by which these requirements are realised.

Chapter 5 Proposed Approach and Methodology.

This chapter explains how the specification of Chapter 4 is realised and how the project is carried out. It declares the overall architectural plan and the rationale of the plan, gives the procedural detail of the mechanisms that control the behaviour of the system, outlines the development methodology and

the sequence of building, and outlines the evaluation methodology. The focus is on the rationale and the processes and not on the implementation detail, which is in Chapter 6.

5.1 Overall Approach

The method is based on one proposition, on which all the structural decisions in this project are based: the limitation of the scope of the assistant should be structural and not behavioural. It should be impossible by design and not discouraged by teaching.

The difference is consequential. A system that instructs a model not to access material outside a workspace has expressed a preference, and that preference can be defeated by a sufficiently persuasive document, an unanticipated phrasing, or a defect in the model's compliance. A system where no other component than a single broker can transform a reference into a filesystem path, and where that broker determines the path to its actual location and then checks containment has asserted an invariant. The former can be argued past, the latter cannot, since there is no code path along which the argument can run.

This suggestion is repeated thrice, which results in the three mechanisms that characterize the system.

The initial use is confinement. All path resolution is directed through a single component, canonicalised and then containment is tested. This is what makes the difference between a boundary and a suggestion, since a path with a traversal segment or a symbolic link to the outside of the grant is solved to its real position prior to the test being run. To undo these two steps would be to accept a course which only seemed to be enclosed.

The second use is the denial of destruction. Instead of putting destructive operations behind a confirmation, the system offers no operation that does one. No deletion approval dialogue exists since there is no deletion code path. A policy is a gate that the user can click through; an architecture is a gate that does not exist. It is what allows the assertion that the system deletes nothing is a property of the system and not a promise concerning the behaviour of a model.

The third use is the separation of proposal and execution. The planner creates a plan and is unable to implement it. The dispatcher performs and is unable to plan. The policy engine between them makes a decision on every proposed call, and the dispatcher rejects any call whose decision is not present or does not match it. It is this separation that renders prompt injection a constrained problem: text in a document cannot be an authorised instruction, since an instruction must be a well-formed declaration that has passed an out-of-band policy check, and no quantity of persuasive prose can generate one.

5.2 The Risk Classification Procedure.

The policy engine categorizes each proposed call into precisely one of four levels. The system has a major decision mechanism, the classification procedure, and its properties are not accidental.

The order of evaluation is predetermined, and denials are considered prior to permissions. Every rule is returned immediately, and thus the decision is the first rule to give what a subsequent rule cannot give. The order is as follows.

1. If any path the call touches lies outside the active grant, deny it as destructive
2. If the operation is itself destructive, deny it
3. If the operation leaves the machine, gate it, and do so regardless of configuration
4. If the operation changes state, gate it unless the user has recorded a rule permitting that operation in that workspace
5. If the operation is a recognised read, allow it
6. Otherwise deny it, because an unrecognised operation is treated as unknown rather than as harmless

The final rule warrants comment. An operation the system does not recognise is refused rather than permitted, because silently permitting unrecognised operations is the mechanism by which a boundary erodes over time. A system that defaults to permission accumulates capability it was never designed to have.

70 The containment test within the first rule is delegated to the broker rather than reimplemented. This is deliberate: it ensures that exactly one containment check exists in the system, rather than two that may drift apart as the code changes.

Table 5.1 states the four tiers, their default treatment and their recoverability.

Table 5.1: The four risk tiers.

Tier	Definition	Default	Recovery
T0 Read	Reads within the granted workspace. No change, no egress.	Executes automatically	Not applicable
T1 Reversible	Changes workspace state and is fully invertible from a snapshot.	Configurable: automatic, batch approval, or per action	Full

T2 Outward	Leaves the machine: mail, web request, cloud model call carrying workspace content.	Always gated. Not configurable.	Partial; logged, and recallable only where the protocol permits
T3 Destructive	Deletion, overwriting without a snapshot, writing outside the workspace.	Refused. Not gated, because structurally unavailable.	Not applicable

The adjustable level is T1 in isolation and this is how the research discovery in Chapter 2 is respected. Since the respondents were almost evenly split on whether an assistant should seek permission before taking action, approval granularity is a setting and not a choice. An approval remembered is limited to a single operation in a single workspace, such that approval of a folder tidy does not grant permission to send mail, and the engine will not remember such a rule to an outward or destructive operation regardless of what is requested.

5.3 The Recovery Procedure

Two procedural rules govern recovery, each of which deals with a common failure mode of undo mechanisms.

The snapshot is taken prior to the mutation, never after. The inverse is recorded post facto, and the state that the operation has already created is reversed to nothing. Each route through the journal thus writes its entry and only then allows the caller to act.

The strategy is selected by cost. A tiny file is replicated into a content-addressed store, such that successive snapshots of an identical file incur no extra cost than the initial. A big file is hard-linked where the filesystem allows it. A relocation or renaming does not involve any copying of content whatsoever, as its reverse is purely structural. A deletion record is sufficient to create a new file.

Reversal is at the granularity of the plan and not the individual operation. A reverse of a folder tidy undoes all the moves of the folder in reverse order since the plan is the unit that the user was requested to approve. It needs to be reversed, not cosmetic: a program that builds a directory and then transfers files into it must transfer the files out of it before the directory can be deleted. A step that cannot be undone is not re-raised as an error, and thus one bad file does not leave the rest of the reversals behind.

5.4 The Prompt Injection Defence

The system reads documents, source code and web results, all of which can be manipulated by an adversary, and the security literature examined in Chapter 3 recognizes this as the main threat to an agentic system. It is not a detection approach, which is unreliable, but the structural impossibility of retrieved content becoming an authorised instruction. There are five independent defences, each of which is adequate on its own..

- Each context chunk is marked with its provenance user, workspace file, web or tool result, and only user chunks can be the source of an intent. All others are treated as data
- The planner emits a structured plan rather than prose, so a tool call must be a well-formed declaration and text within a document cannot become one by being persuasive
- The policy engine does not read the reasoning text of the model, only the tool call, and therefore nothing in a document can change a verdict.
- Each outward action gives the user a tangible preview of what will happen, recipient and content, and thus an attempted exfiltration must pass through a person reading it in plain text.
- The workspace boundary restricts the effects of a completely successful injection to a single granted workspace, and no deletion or egress outside the policy of that workspace.

The final defence is the strongest argument for the bounded-scope hypothesis, because it bounds the worst case rather than only the expected case.

5.5 Development Methods

The project is developed in an incremental and iterative approach structured into two-week sprints, and the backlog is represented in the form of user stories with Fibonacci point estimates. Every sprint provides a single step in the build sequence and ends with a stage gate, a test that must pass before the stage is declared complete. A stage is not delivered because its code is; it is delivered because its gate is passed.

The decision to use an incremental approach is based on the risk profile of the project and not preference. The main danger is that the trust machinery is impossible to retrofit when features rely on it, and the approach taken is to build that machinery first and test it on inert tools before any model is engaged.

There were two other approaches. A waterfall approach was dismissed since the specification relies on discoveries that can only be made during construction, including whether a local model of workable size can plan sufficiently. An entirely feature-based model, constructing the visible capabilities and

then adding protection later, was dismissed on the more compelling ground: the boundary and the gates are the very elements that cannot be added afterwards, since all the features constructed before them would have to be rewritten to traverse them.

The build sequence is shown in Table 5.2. Its ordering is a single principle, that a property must exist before anything can be demanded to possess it..

Table 5.2: The build sequence.

Stage	Deliverable	Reason for this position
1	Workspace foundation: the store, the workspace broker, the file watcher and the disclosure panel. No model involved.	The boundary must exist before anything can be bounded.
2	Policy engine, approval broker and undo journal, exercised against placeholder tools.	Inexpensive now and near-impossible to retrofit; provable before any model introduces non-determinism.
3	The gateway: model router, context engine, agent planner and tool dispatcher.	The highest-risk stage, validated against gates that already function.
4	Session resume, folder tidy and repeated task detection.	The first real capabilities, exercising the read and reversible tiers end to end.
5	Notes synthesis and code assistance, with the index, retrieval and citation map.	Introduces the vector index only once the boundary is proven.

Table 5.2: continued from previous page

Stage	Deliverable	Reason for this position
6	Web search, mail composition and document drafting.	The first outward actions, permitted only after the gates are demonstrated.
7	Speech input and output over the existing turn normaliser.	Should require no new gate. If it does, the interaction layer was designed wrongly, so this stage tests the parity claim.
8	Remote access plane.	Genuinely optional; a complete product exists without it.

The most consequential decision in this sequence is the placement of stage 2 before the model work. It implies that the trust machinery can be tested, and proved, prior to any model non-determinism being introduced into the system. A defective gate at that point is a deterministic defect with a short reproduction, which is no longer true when a model is producing the plans.

The stage 7 gate is of special interest since it is designed to fail. The assertion that voice is parity by construction is challenged by the fact that the speech stage must not add a new approval gate. In case a new gate was needed, the assertion would be incorrect and the interaction layer would require redesign. A gate that fails no tests.

5.6 Evaluation Methodology

The assessment, which is performed in the second stage of the project, is a test of the central hypothesis and the constraints that the literature review found to be inherited. It consists of four elements.

Task-based measurement involves comparing the participants who complete representative multi-file tasks with and without the system, and the time and success rate of completion. The cognitive workload is assessed with a developed subjective measure that is given at the end of each task. The measurement of usability and trust is by questionnaire, and trust is instrumented against the bounded-scope claim, such that the hypothesis is tested and not assumed.

Three other tests deal directly with the inherited limitations, since the literature suggests that each would otherwise be asserted rather than proved. Approval fatigue is not tested in a single session, but in repeated sessions, as it cannot be observed in a single session. Synthesis faithfulness and citation accuracy are quantified wherever source metadata allows, and any deficiency is reported as a known limitation. Code assistance is checked to ensure that no change is ever made without express permission and that no previous user edit is overwritten without notice.

At the end of the first phase the parts of this methodology that do not require participants were carried out and are reported in Chapter 9, and the parts that do are planned for the second phase and are listed in Chapter 10.

The voice assessment is based on the design advice provided in the literature. It sets a target of four seconds latency threshold, natural spoken fillers instead of tones during waiting times, and reports cognitive workload and trust and task time and success, as the reviews determined that standard usability measures do not capture these voice-specific issues.

5.7 Conclusion

The approach and the methodology have been outlined in this chapter. It said the plan of laying the guarantees in structure, provided the steps to classify the risks, recover and the defence against prompt injection, explained the incremental approach and the build sequence where each step has to pass its gate before the next one starts and the evaluation. The steps outlined here are the steps that the implementation takes and the evaluation tests. The next chapter presents the design that realises them.